



Fara-1.5: Scalable Learning Environments for Computer Use Agents

Ahmed Awadallah, Sahil Gupta, Yash Lara, Yadong Lu, Hussein Mozannar, Akshay Nambi, Zach Nussbaum, Yash Pandya, Aravind Rajeswaran, Corby Rosset, Alexey Taymanov, Luiz do Valle, Vibhav Vineet, Spencer Whitehead, Andrew Zhao

 <https://aka.ms/fara1.5>

 <https://github.com/microsoft/fara>

 <https://aka.ms/fara1.5-hf>

 <https://aka.ms/fara1.5-9B-foundry>

Collecting computer use data from human demonstrations is expensive and slow, motivating the need for scalable generation strategies. This requires two key ingredients: *environments* in which agents can act and *verifiers* that can judge whether their demonstrations succeeded. We introduce FaraGen1.5, a scalable data pipeline for computer use agents composed of three modular components: environments, solvers, and verifiers. FaraGen1.5 uses both live websites and synthetic environments that faithfully simulate domains gated by authentication or that require irreversible actions. It employs a solver harness that can be powered by multiple models, including strong frontier models such as GPT-5.4, and also incorporates a user simulator to enable multi-turn rollouts. Finally, FaraGen1.5 scores the resulting trajectories with three complementary verifiers covering task correctness, efficiency, and critical-point adherence. Using data produced by this pipeline, we train Fara1.5, a family of native computer use agents (CUAs) at three scales built on Qwen3.5 (4B, 9B, and 27B). To train these models, we employ a supervised finetuning (SFT) recipe that carefully balances data from FaraGen1.5 for broad coverage, specific high-value tasks, and target model deficiencies in an iterative approach. Each model sets a new state of the art (SoTA) for its size class on browser-use benchmarks: Fara1.5-9B reaches **63.4%** on Online-Mind2Web and **86.6%** on WebVoyager, while Fara1.5-27B achieves **72.3%** on Online-Mind2Web, which is competitive with much larger proprietary systems. We also release weights for the Fara1.5 models under MIT license, making SoTA computer use accessible for all beyond closed API-only systems.

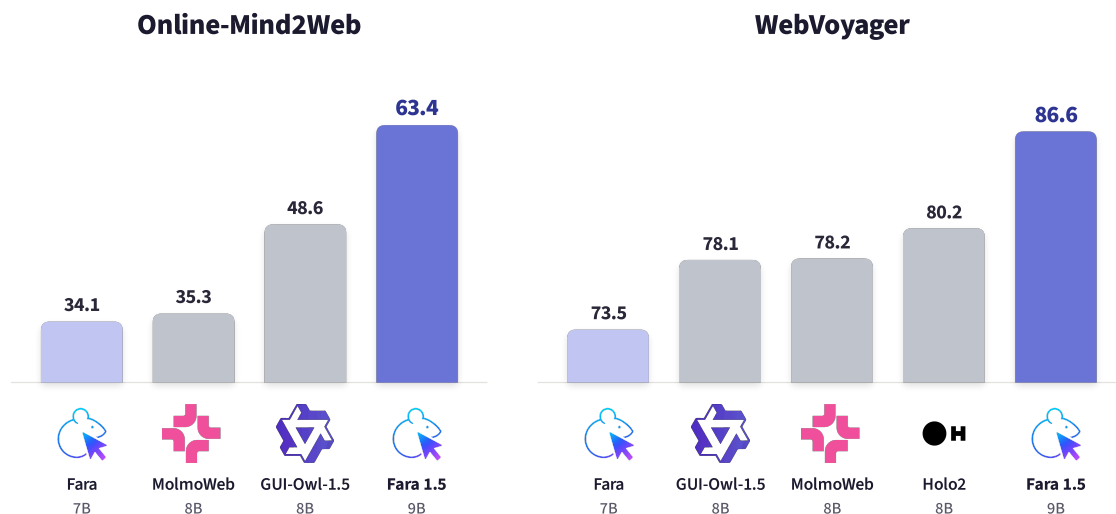


Figure 1: Task success rate on Online-Mind2Web and WebVoyager for similarly sized CUA models. Fara1.5-9B reaches 63.4% on Online-Mind2Web (a +29.3 point improvement over Fara-7B (Awadallah et al., 2025)) and a +14.8 point improvement over the prior best at this scale, GUI-Owl-1.5-8B) and 86.6% on WebVoyager, setting a new state of the art for the 8–9B size class on both benchmarks.

1 Introduction

Large Language Models (LLMs) are increasingly being deployed as agents that act on a user’s behalf, with computer use agents (CUAs) for web browsers emerging as an immediate and impactful instantiation (Anthropic, 2024; Awadallah et al., 2025; DeepMind, 2025; OpenAI, 2025c). Users provide tasks in natural language and the agent carries out the necessary steps like filling out forms on their computer. Training such agents requires computer-use demonstrations of tasks on real world digital environments and verifiers to ensure the demonstrations accomplish the task. This type of data does not exist freely on the internet and human demonstrations are too expensive and slow to collect, motivating the use of automated recipes to generate such data (Awadallah et al., 2025).

Live websites provide rich, real-world environments for CUAs to operate in, but they also carry real-world constraints and risks. A wide range of user tasks are time-sensitive in nature, meaning they quickly become invalid with natural progression of time or website changes. Another challenge is that many tasks require credentialed login for completion or task verification, which is dangerous for agents to attempt without oversight, especially for under-trained models without guardrails. Finally, live websites may also have policies that limit the movement of agents or bots through them. These practical constraints put limitations on the ability to scale training data from live websites.

Meanwhile, recent advancements in CUA research have produced agents that achieve non-trivial success rates on web tasks (Anthropic, 2026; OpenAI, 2026). These models are often proprietary systems with complex scaffolds that allow an LLM to act as a web agent. Nevertheless, they can serve as useful teacher models for web data collection, to be ultimately distilled into simpler “*native*” CUA agents that consume pixels and produce actions (Awadallah et al., 2025). For these trajectories to act as useful training data, a high-quality *verifier* is necessary to ensure quality control. This is a difficult task in itself as fully programmatic verification may not be feasible for websites with inaccessible backends, and LLMs as judges may hallucinate.

In this work, we approach these challenges in two ways. First, building on our previous efforts for Fara-7B (Awadallah et al., 2025), we improve our data generation pipeline by redesigning our verifiers so they are more reliable as well as streamlining our task solving system and incorporating stronger teacher models. Second, we generate *synthetic environments*: fully functional and sandboxed websites, complete with frontend and backend infrastructure. These synthetic environments allow us to create specific scenarios and tasks that go beyond the open internet, such as checking emails or managing experiments from a cloud provider. To support these new kinds of tasks, we refine our approach for generating synthetic user interaction data, allowing us to create trajectories that involve realistic exchanges with simulated users. Additionally, in these synthetic environments we have full control over the website and we are able to verify trajectories based the actual state of the website, rather than only screenshots or other artifacts. These advancements make up the next iteration of our data generation pipeline, FaraGen1.5.

Results. We train Fara1.5, a new family of CUA models for web browsing, using our improved FaraGen1.5 pipeline. This family of models spans three sizes (Fara1.5-4B, Fara1.5-9B, and Fara1.5-27B) and are the most capable web agents for their respective sizes while remaining practical to deploy on modest hardware (Figure 1). Compared to our prior Fara-7B generation (Awadallah et al., 2025), we see clear improvements both quantitatively across all benchmarks and qualitatively through user experiences. Across key benchmarks, Fara1.5 outperforms other models of similar sizes. For example, on the **Online-Mind2Web benchmark** consisting of 300 tasks across 136 popular sites, Fara1.5-9B achieves a task success rate of **63.4%** which nearly doubles the performance of Fara-7B and significantly improves over the performance of GUI-Owl-1.5-8B (48.6%), the prior best performing model at this scale. The larger Fara1.5-27B model achieves **72.3%** on the same benchmark, which is competitive or outperforms much larger models and proprietary systems like Gemini 2.5 Computer Use (DeepMind, 2025) and Yutori n1 (The Yutori Team, 2025).

We also train Fara1.5 with a keen eye on improving user experience. This includes the choice of task distribution, synthetic domains we designed, the user-interaction pattern, adherence to user preferences and constraints. Unlike the predecessor Fara-7B, which was trained to never go past critical points – irreversible actions like final steps in making payments or submitting forms – Fara1.5 is trained to complete those steps after explicitly seeking out user approval. We complement our training data with synthetic domains that

FaraGen1.5: End-to-End Synthetic Data Generation for CUA

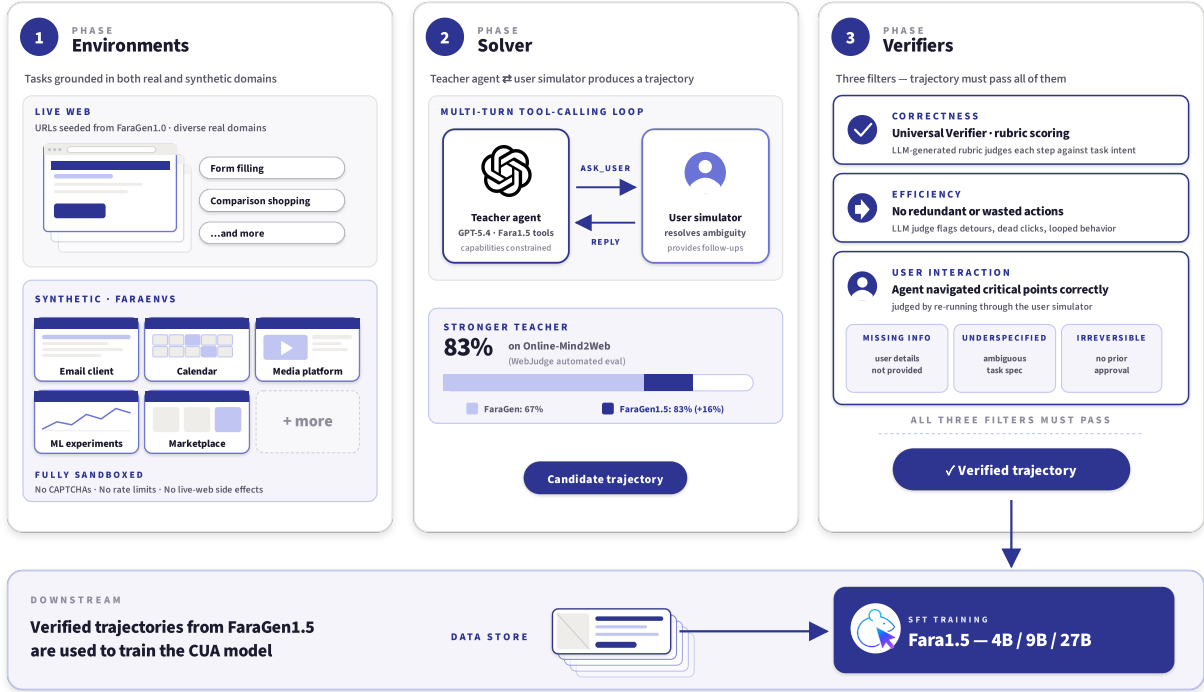


Figure 2: FaraGen1.5 pipeline. Phase 1 instantiates tasks in two kinds of environments: *online*, seeded by URLs from the FaraGen (Awadallah et al., 2025) index, and *synthetic*, a set of six sandboxed replicas (*FaraEnvs*) covering domains gated by authentication. Phase 2 has a GPT-5.4-based solver attempt each task in cooperation with a user simulator; the new solver reaches 83% on Online-Mind2Web under the WebJedge of Xue et al. (2025), a +16-point absolute gain over the multi-agent FaraGen solver. Phase 3 admits a trajectory into training only if it passes all three independent verifiers: correctness, efficiency, and critical-point adherence.

simulate popular online websites/apps to allow our model to act beyond gated domains and, e.g., send the email or book the flight rather than just searching for it. As a result, on internal user studies, we found that users experience smoother interactions and better control over their tasks.

Open release. The Fara1.5 family is released as open-weight models on HuggingFace, along with a model card and usage guidance. We hope this enables widespread adoption, reproducibility, and community research on computer use and web agents.

2 FaraGen1.5 – A Data Engine for Computer Use Agents

Training performant CUAs is bottlenecked by the lack of high-quality interaction data. Human demonstration data is slow and expensive to collect and there is no naturally occurring dataset capturing realistic multi-step tasks. Additionally, existing synthetic pipelines are limited to what can be accomplished on the open internet, without authentication or irreversible side effects. FaraGen (Awadallah et al., 2025) generates verified trajectories from live websites at roughly \$1 per task, but the open web limits the activities a downstream agent can learn. The agent never observes what successful login flows look like, how purchases are confirmed, or how messages get sent. FaraGen1.5 is the next evolution of this pipeline designed to address these challenges. As shown in Figure 2, we retain the three-stage organization of FaraGen: environments that instantiate tasks, a solver that produces trajectories to solve tasks, and verifiers that filter the trajectories. However, we rebuild each stage to address specific challenges: the open-web restriction on environments, the plateau in solver quality, and the limited reliability of single-perspective verification.

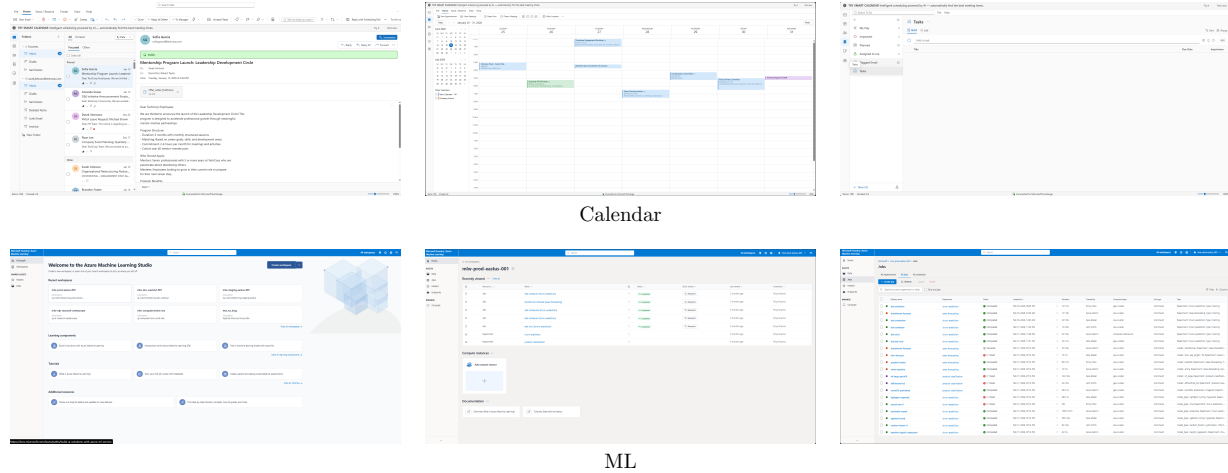


Figure 3: Sample FaraEnvs for Calendar and ML Management environments populated with realistic data generated by our pipeline, illustrating rich, production-like workflows and interactions across both environments.

2.1 Environments

2.1.1 Live Web Environments

Live web environments cover tasks that can be completed on the open internet safely and without authentication. We source a wide range of websites to use as seeds for task generation from a large index of publicly accessible URLs (Overwijk et al., 2022). To generate tasks for these sites, we first create summaries of each seed website’s content, features, and structure through an initial exploration of the site which is then cached and re-used. We provide this summary as part of a prompt to an LLM, which uses the summary to generate plausible tasks for the website.

We propose tasks by sampling along several dimensions of variation: target website, task complexity, phrasing style, user interaction type and task feasibility. Given this sampling, we feed the values and the website summary to a task proposal LLM. This proposal LLM is instructed to select a user intent (e.g., form completion) and persona (e.g., busy professional, student) that is plausible for the target site from a pre-defined set, and generate tasks based on the combination of the intent, persona, sampled dimensions, and website summary. This hybrid scheme balances controlled coverage along the policy-relevant dimensions with site-specific plausibility along the user-side ones. Since these dimensions are categorical, we use a coverage tracker to monitor the empirical distribution along every dimension. In Table 8 we show the different sampling dimensions for tasks, and Table 9 shows a sample of random tasks from this pipeline.

We put the proposed tasks through a two-stage filter. A deterministic check rejects surface-level failures (e.g., LLM refused, uses bare URLs), and an LLM judge then rates each remaining task as good or bad on whether it describes something real with a clear goal. Intentionally ambiguous or under-specified tasks are kept if they are judged as realistic by the LLM, since this matches the user-instruction distribution we want our solver to handle.

This process yields a large, rich set of tasks for a broad set of URLs and types of tasks. However, as previously mentioned, the distribution of tasks covers those that can be completed on the open internet. We build synthetic environments to target tasks beyond these.

2.1.2 Synthetic Environments

Synthetic environments cover tasks that online environments cannot: tasks behind authentication, tasks that mutate persistent state, and tasks where the only success criterion is that an irreversible action was taken correctly. The defining property of a synthetic environment is that we control the full stack - frontend, API, database, and seed data - so every task has a verifiable ground-truth success criterion expressible as a state

predicate over the backend. This yields two key properties. First, tasks with irreversible side effects can be attempted safely: the websites are sandboxed and the database is reset between trajectories. Second, we can use the internal website state for execution-based verification of trajectories, rather than relying solely on screenshots or other surface metadata.

Hand-authoring high-fidelity replicas of complex web apps does not scale to many domains, so we instead use a semi-automated recipe driven by a coding agent (GitHub Copilot ([GitHub, 2026](#))). We first record human interaction trajectories on the target domain. These trajectories are given to the coding agent, which generate a specification for a sandboxed clone consisting of a React frontend, a FastAPI backend, a SQLite schema, and a seed-data script to populate the database from a persona narrative. The coding agent iteratively refines the clone through several rounds of human review: the first iteration is typically incomplete (non-functional buttons, missing edge cases, inconsistent state transitions), but each round of feedback continually improves the website. After a small number of iterations, typically 3-5, we obtain a replica whose surface behavior matches the recorded human trajectories. This human-agent loop accelerates the development process while ensuring that the resulting environments are functionally correct.

Once an environment has been created, we generate tasks via a task proposal LLM that has direct access to the hosted synthetic environment through Model Context Protocol (MCP) tool servers: a SQLite MCP server that introspects the schema and queries entities and a Playwright MCP server that browses the live UI. The LLM is also given a small bank of human-written exemplar tasks per category to anchor style as well as a user persona, similar to our live web environments. For example, in our email environment, the database is populated as the inbox of an employee at a small IT firm, with internally consistent calendar invites, project threads, and recurring collaborators. Tasks are proposed in two separate phases for the training split and a held-out test split, which are deduplicated against one another so no tasks/intents are overlapping. Since the backend state is fully observable, every task carries a precomputed success criterion. For tasks that mutate state, we record a database snapshot before and after the trajectory and use `sqldiff` to compute the row-level difference, which an LLM judge confirms is the intended mutation based on the task. For read-only tasks, we precompute a reference answer (or a SQL query that produces it) at generation time and score the trajectory’s final response against it.

Environment diversity matters more than data volume. Recent work ([Wang et al., 2026](#); [Aggarwal et al., 2026](#)) that expand to more trajectories within the same environments cannot substitute for it, and shallow environments add little value regardless of how many trajectories they yield. Using our pipeline, we produce six synthetic environments (Figure 3) that we collectively call *FaraEnvs* – email, calendar, media streaming, ML experiment management, marketplaces, and scheduling – that prioritize **depth**: environments populated with coherent, realistic data; environments whose tasks require understanding the application’s actual data model and UI patterns; and environments that feel like real user workflows rather than simple tasks.

2.2 Solver

The solver attempts each proposed task and returns a trajectory that, if accepted by the verifiers, becomes a training example. In the original FaraGen, the solver is an orchestrator-worker multi-agent system built on Magentic-One and Magentic-UI ([Fourney et al., 2024](#); [Mozannar et al., 2025](#)). Here we collapse it to a single agent built on GPT-5.4 that runs a multi-turn tool-calling loop over a tool set mirroring the action space the student will eventually emit. Our motivation for this is twofold: 1) A single-policy solver better matches the inference-time setting of the trained Fara1.5 models, reducing distribution shift between teacher and student; 2) it lets us inherit improvements in frontier model capability without re-engineering the orchestration layer for each generation of model. On the same Online-Mind2Web tasks ([Xue et al., 2025](#)), replacing the multi-agent system with a GPT-5.4-based solver raises end-to-end success from 67% to 83%, a +16-point absolute gain that translates directly to a higher yield of accepted training trajectories. To keep this data *learnable* by the smaller Fara1.5 students, we explicitly disable solver capabilities the student cannot replicate. In particular, within our harness, we forbid the solver agent from issuing complex URL queries that would bypass site interaction entirely, since these queries are harder for smaller models to replicate and the resulting trajectories skip the UI interaction behavior we want the student to learn. The solver is also not allowed to do any dangerous actions or irreversible actions on the web such as creating accounts or posting on forums.

Permission granted	Fully specified	PII provided	Example	Expected agent behavior
×	×	×	Please book me a flight at [url].	Collect the required fields, ask the user to provide them, and fill them in; repeat for each page, then ask for permission and submit only if it is granted.
×	✓	×	Book the Standard room at [url] for check-in Dec 20th, check-out Dec 25th.	Fill in the already-provided information, then ask for and fill in any missing required fields; repeat for each page, then ask for permission and submit only if granted.
×	×	✓	Sign me up at [url]. I'm [name], [email], phone [phone].	Fill in the already-provided information and ask the user to supply anything still missing; repeat for each page, then ask for permission and submit only if granted.
×	✓	✓	Register me for the morning Python workshop on March 15th at [url]. Name: [name], Email: [email], Phone: [phone].	Fill in all provided information across every page, then ask for permission and submit only if it is granted.
✓	×	×	Please go to [url] and book me a flight. You have my permission to submit the purchase.	Fill in the already-provided information, then ask the user for the missing required fields and fill them in; repeat for each page, then submit.
✓	✓	×	Register me for the morning Python workshop on March 15th at [url]. Submit when complete.	Fill in the already-provided information and advance through every page, then submit. [†]
✓	×	✓	Sign me up at [url]. I'm [name], [email]. You can submit it.	Fill in the already-provided information and ask the user to supply anything still missing; repeat for each page, then submit.
✓	✓	✓	Fill out [url] with my info: [name], [email], [phone]. Select 'Technical Support' as the topic. Submit it.	Fill in all provided information across every page, then submit (no pause required).

Table 1: The eight critical point types, one per combination of the three dimensions defined in the text: **Permission granted** (the user authorized the irreversible action), **Fully specified** (all task-specific information was provided), and **PII provided** (all required personal information was provided). A ✓ means the condition is satisfied; a × means it is not and marks where the agent must pause and consult the user.

User simulator. As the solver is attempting a task, a user simulator is invoked in two distinct circumstances. First, when there is missing information for the task or the task is ambiguous, the solver agent issues an ask_user and the simulator responds with the necessary information to continue. Second, once the solver believes a task is complete, the simulator can either accept the result or extend it with a follow-up request, turning a single-task demonstration into a multi-turn dialogue; this second mode is the principal source of multi-turn training data for Fara1.5. The exact prompts used at both callsites are given in Appendix A.4.

Because the solver runs against real, live third-party websites, an unrestricted user simulator could push it into actions with real-world consequences — creating accounts, completing checkouts, sending messages, or submitting real personal information — all of which mutate third-party state and create legal exposure. We therefore guard the ask_user callsite with a lightweight LLM gate that fires before the simulator drafts its reply. The gate inspects the agent’s pending question against a list of forbidden side-effect categories (account registration, real login, real checkout, real messaging, public posting, subscription changes, real-data mutation, identity-tier PII, and final booking steps) and only lets the simulator answer when the question is in the harmless filtering, sorting, or preference-selection regime. Otherwise the trajectory is halted and dropped. Gate decisions, including the reasoning trace, are persisted alongside the trajectory so we can audit and tune the boundary over time.

2.3 Verifiers

A high-quality solver still produces a long tail of low-quality trajectories: tasks completed in the wrong way, tasks completed inefficiently, or tasks where the agent should have deferred to the user but did not. We use

three independent verifiers, each targeting a distinct failure mode, and admit a trajectory into training only if it passes all three. The verifiers are complementary: correctness alone cannot distinguish efficient from meandering successes, efficiency alone cannot catch trajectories that hallucinate a result, and neither catches trajectories that fabricate user information they should have asked for.

Task correctness. We instantiate task correctness differently depending on whether ground truth is available. On live web environments, we use the process-reward score from the Universal Verifier (Rosset et al., 2026), an ensemble LLM judge that generates a task-specific rubric and scores the trajectory step-by-step against it. We accept trajectories whose rubric process score is at least 0.8. On synthetic environments, we use two different criteria depending on the tasks: 1) state-mutating tasks are scored by computing the row-level `sqldiff` between the seed database and the post-trajectory database and asking an LLM judge whether the observed diff matches the task’s intended mutation (and nothing else); 2) read-only tasks are scored by comparing the trajectory’s final answer to a precomputed reference. The judge for state-mutating tasks has lower label noise than the Universal Verifier, which is one reason we treat synthetic environments as a strict generalization of online ones; when a task *can* be expressed synthetically, we prefer it.

Task efficiency. A correct trajectory is not automatically a good training example. Agents frequently complete tasks that contain redundant clicks, unnecessary navigations, or repeated state-checks. Imitating these behaviors yields a student that is correct but slow, with all the resulting cost and latency consequences for inference. The task efficiency verifier is a dedicated LLM judge that explicitly identifies looped action sequences as well as individual unnecessary actions, and assigns each trajectory an integer efficiency rating on a 1-5 scale. We accept trajectories whose rating is at least 4.

Critical-point adherence. The third verifier targets behavior at *critical points*: steps at which the agent should pause and consult the user before acting. We first classify each task along three dimensions:

1. *Permission to perform irreversible action*: Does the task require an irreversible action and did the user explicitly provide authorization to perform it? Examples include finalizing a purchase or submitting a contact form.
2. *Task fully specified*: Was all the task-specific information needed to complete the task provided? For example, booking a flight would require the departure and return dates.
3. *Required user information provided*: Did the user provide all required Personally Identifiable Information (PII) needed to complete the task? For example, signing up for a newsletter would require the user’s email address.

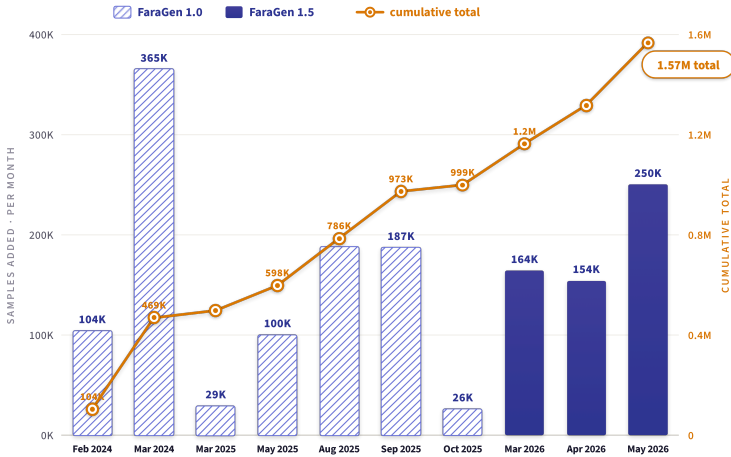
These three dimensions combine to produce 8 distinct types of critical point classifications (Table 1). The classification is done by an LLM judge and is based on the initial task prompt and the screenshots of the pages that the agent interacted with (to identify required fields, for instance). During verification, the classification is injected into the rubric that scores the trajectory, and a dedicated compliance check determines whether the solver actually exhibited the expected behavior at each critical point. Trajectories that cross a critical point without prior approval are rejected. This verifier enables our model to know when to appropriately defer to the user.

2.4 Data Distribution

We have run FaraGen1.5 since the first generation of the pipeline (Figure 4, left), accumulating roughly 1.57M trajectory steps by May 2026. Web trajectories from FaraGen1.5 remain the dominant component of the final training corpus at 60.0%, with synthetic-environment trajectories contributing 12.8% and form-filling and user-interaction trajectories contributing another 12.5%. Together these three FaraGen1.5-sourced categories make up 85.3% of the Fara1.5 training mix (Figure 4, right). We discuss the other main auxiliary data below.

Grounding. This is the task of locating a UI element from a natural-language query against a screenshot, and is a fundamental sub-task for CUAs. We add data from Jedi (Xie et al., 2025), GroundCUA (Feizi et al., 2025), and Click100k (Awadalla et al., 2025). These open datasets are not uniformly high quality (see Section A.8 for failure-mode examples), so we filter each example with GPT-5.2 along four criteria: *uniqueness* (does exactly one UI element satisfy the query?), *relevance* (does the screenshot show the queried interface?), *clarity* (is

Agentic training data (steps) collected over time



Fara1.5 Training Data Mix

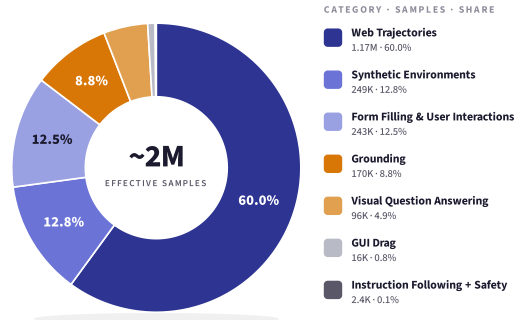


Figure 4: (Left) Agentic training data collected by FaraGen1.5 and its predecessor over time, reported as the number of trajectory *steps* added per month and the running cumulative total, which reaches 1.57M steps by May 2026. (Right) Final Fara1.5 training data mix, dominated by web trajectories (60.0%) and complemented by synthetic-environment trajectories (12.8%), form-filling and user-interaction trajectories (12.5%), grounding (8.8%), visual question answering (4.9%), GUI drag (0.8%), and instruction following plus safety (0.1%); the auxiliary data mix is discussed further in Section 3.2.

the query unambiguous about which element it refers to?), and *accuracy* (does the ground-truth coordinate actually lie on the described element?).

Safety. To teach the model safe behaviors on harmful tasks, we add two kinds of refusal data. The first is grounding-style refusal data from Jedi, constructed by pairing open-source instructions with unrelated screenshots so that the correct response is to decline. We filter this data with GPT-5.2 to retain only examples where the instruction is genuinely not executable against the paired image. The second is task-level refusal data seeded from the harmful tasks in WebTailBench-Refusals (Awadallah et al., 2025): we use an LLM to generate variants of each seed task that cover the same harm categories with new wording and context, so that the model learns to refuse based on the nature of the request rather than memorizing specific phrasings.

Image understanding. Finally, to bolster the model’s ability to extract information from screens and avoid hallucinations, we add visual question answering data from open sources including RICO-SCA (Li et al., 2020) and InfographicsVQA (Mathew et al., 2021).

3 Fara1.5 – A Family of Native CUA Models

We train Fara1.5 on trajectories produced by FaraGen1.5, distilling the solver’s behavior into a family of small CUA models at three scales (Fara1.5-4B, Fara1.5-9B, Fara1.5-27B). Each model operates as a single native policy: a VLM that consumes screenshots and emits low-level actions in a tight observe-think-act loop.

3.1 Formulation

Our setup largely resembles Fara-7B. Given an initial natural language user query, Fara1.5 generates a trajectory step-by-step until it terminates (Figure 5). At each step, the model takes in the current browser state as observation, produces a chain-of-thought reasoning trace that reflects on the state and what to do next, and emits a single atomic action. The action is executed in the browser, the browser updates, and the cycle repeats. We train the model to predict the next thought and action given the user’s query and the full history of preceding observations, thoughts, and actions. This formulation also supports follow-up interactions: if

Fara1.5: Observe–Think–Act Loop

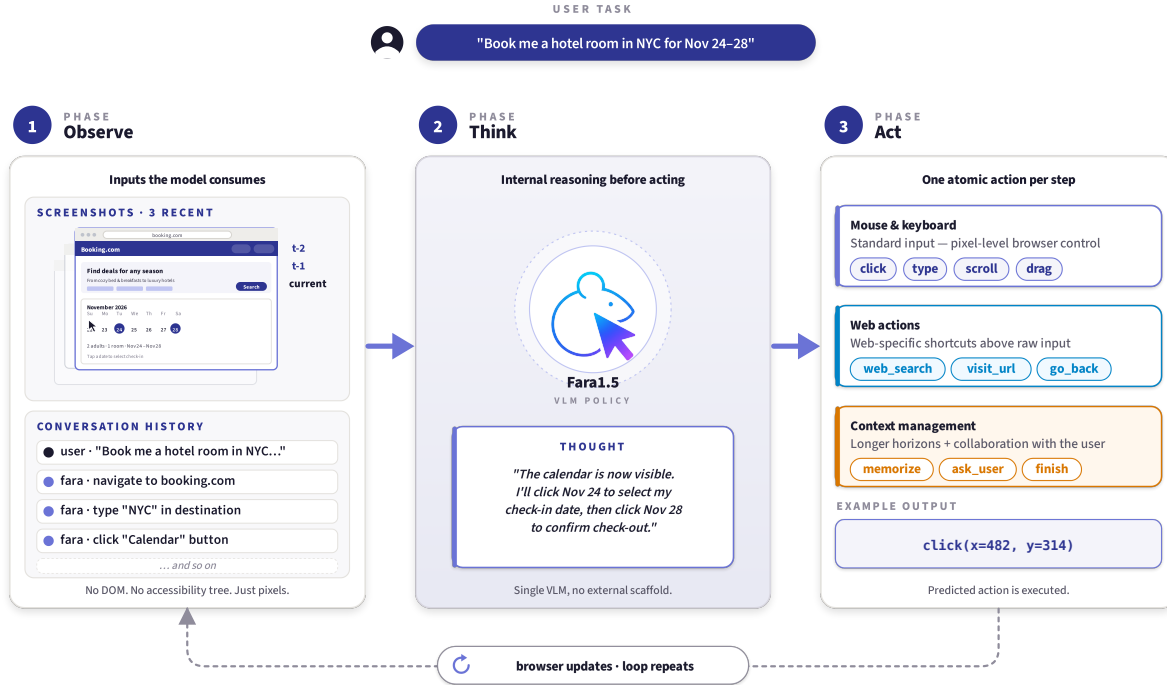


Figure 5: One step of Fara1.5’s observe-think-act loop. The model observes the recent browser state (up to three screenshots plus the conversation history), reasons internally about the next step, and emits a single atomic action. No DOM, accessibility tree, or external scaffold is consumed at inference time.

the user issues a new query after the model has finished, we simply append the query to the history and continue. This is what allows the user simulator’s follow-up requests (Section 2.2) to translate directly into multi-turn training data.

Rather than using DOM serializations or accessibility trees (Abuelsaad et al., 2024; GLM-V, 2025; He et al., 2024a; Zhou et al., 2023), Fara1.5 observes only the rendered screenshot along with a short *Current URL* prefix added at each step. We trim the URL to one hundred characters and strip query parameters to keep it compact. The URL is the only structured browser metadata in the input. It is cheap in tokens, helps disambiguate visually similar pages (e.g., different product pages on the same retailer), and gives the model a stable anchor for cross-page reasoning. The model predicts normalized coordinates in pixel space.

Our models first output a thought describing its intent and any additional useful information, such as the content of the page, followed by an action. The full action space is listed in Table 2, where actions are represented as tool calls. Beyond standard mouse-and-keyboard (e.g., `left_click`, `keypress`) and browser-specific actions (e.g., `visit_url`), we also include three meta-actions. The `pause_and_memorize_fact` action lets Fara1.5 store an intermediate fact (e.g., a price quoted on an earlier page) for later use in the same trajectory. This is important when key pieces of information needed for the task live on different pages (e.g., comparing specs of an item between retailers). The action “`ask_user_question`” yields control back to the user at a critical point. This allows the user to authorize the action, provide missing information (e.g., an address if needed), or make changes to the agent’s proposal. Finally, the `terminate` action signals the end of the trajectory and returns the final response. Two end-to-end trajectories produced by Fara1.5-9B that illustrate this loop are in Appendix A.6.

To manage tokens in the context window, we follow Fara-7B (Awadallah et al., 2025) and only keep the most recent observations in the context. However, all previous thoughts and actions are preserved. After

Action	Description
left_click	Click the left mouse button at coordinate (x, y) .
double_click	Double-click at coordinate (x, y) .
right_click	Right-click at coordinate (x, y) .
triple_click	Triple-click at coordinate (x, y) .
left_click_drag	Press the left mouse button at the current cursor position and drag to (x, y) .
mouse_move	Move the cursor to hover over coordinate (x, y) .
type	Type the given text at the current cursor focus.
key	Press a key combination in the order specified (e.g., CTRL+C).
scroll	Scroll the page vertically by a number of pixels (positive for up, negative for down).
hscroll	Scroll the page horizontally by a number of pixels.
visit_url	Navigate the browser to a specified URL.
web_search	Issue a search query and load the results page.
history_back	Go back to the previous page.
pause_and_memorize_fact	Store a piece of information (e.g., a price quoted earlier) for use later in the trajectory.
ask_user_question	Pause the trajectory at a critical point and ask the user for input.
wait	Wait a specified number of seconds for the page to settle.
terminate	End the trajectory and return the final answer to the user.

Table 2: Actions Fara1.5 can take, grouped into pointer and keyboard actions (top), browser navigation actions (middle), and meta-actions (bottom).

experimenting with multiple image context windows, we found that using three images leads to the best tradeoff in agentic task performance and input token consumption.

3.2 Model Training

We train each Fara1.5 model from Qwen3.5 via Supervised Fine-Tuning (SFT). Its grounding and reasoning capabilities provide a stronger starting point compared to language only base models. We train each member of the Fara1.5 family from the corresponding Qwen3.5 backbone at 4B, 9B, and 27B parameters, holding the training data fixed across scales so that downstream comparisons isolate the effect of model size. Full training-time hyperparameters (optimizer, schedule, parallelism, image-processor budgets, compute) are reported in Appendix A.5.

As with Fara-7B, for trajectory data, we treat each individual step of a trajectory as a training sample, providing the history of observations, thoughts, and actions up to the current step as input. We train the model to predict both the thought and the action tokens. Following Qwen3.5, all outputs are tokens from the model’s vocabulary, including the coordinates for grounded actions. Figure 6 shows the loss masking procedure, where we only backpropagate the loss for actions that respond to observations in the context. Both training and inference happen in “instruct” mode despite Qwen3.5 being a hybrid thinking model. We found in practice that enabling thinking at inference time for agentic web tasks adds substantial input tokens to the trajectory with minimal to no performance gains. We hypothesize that this is because the short thought traces generated as part of FaraGen1.5, and used in our training, are superior to the self-generated thought traces of Qwen3.5.

The core of Fara1.5’s training data consists of full trajectories produced by FaraGen1.5 that pass all three verifiers (Section 2.3). We additionally train on data from related tasks that complement agentic computer use: grounding, visual question answering, GUI drag, instruction following, and safety refusal. The final mixture is shown in Figure 4 (right). These ratios have been determined by sweeping over mix proportions and picking the recipe that maximizes agentic accuracy without regressing core capabilities like grounding and VQA. We also found intent driven phrasings of these auxiliary tasks to provide better positive transfer to agentic tasks. For example, we found it beneficial to train on single-step grounding tasks like, “task: increase the quantity by one” which the model maps to “thought: I need to click on the plus icon to increase quantity” and “action: click on <x,y>” instead of directly training on “task: click on plus”.

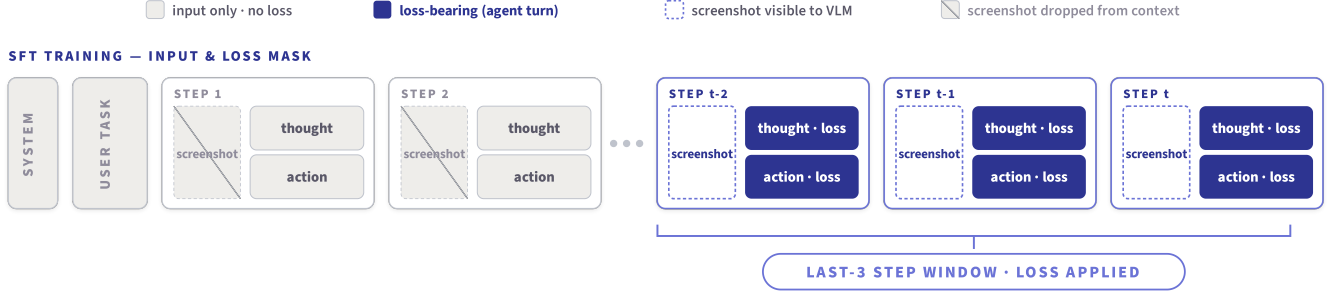


Figure 6: Supervised fine-tuning input and loss mask. The model conditions on the full conversation history but consumes screenshots only from the three most recent steps; cross-entropy loss is applied to the thought and action tokens of those last three turns. Older screenshots are dropped from the input to keep context length bounded.

All models were trained on a cluster of nodes, each containing 8 B200 GPUs interconnected via NVLink. The Fara1.5-9B model was trained using 32 B200 GPUs (4 nodes) for approx. 4 continuous days while Fara1.5-27B was trained using 64 B200 GPUs (8 nodes) for approx. 5 days.

4 Experiments

We evaluate Fara1.5 on agentic browser tasks, grounding, and safety. For agentic tasks, we use three live-web benchmarks: WebVoyager (He et al., 2024a), Online-Mind2Web (Xue et al., 2025), and our own WebTailBench v1.5 (Section 4.1.1).

We compare Fara1.5 models to other CUA models in the same size class: Fara-7B (Awadallah et al., 2025), MolmoWeb (Gupta et al., 2026), Holo2 (Company, 2025), and GUI-Owl-1.5 (Xu et al., 2026). We also evaluate GPT-5.4 (OpenAI, 2026) as a frontier reference, and report leaderboard numbers from larger proprietary systems including OpenAI Operator (OpenAI, 2025c), Google Gemini 2.5 Computer Use (DeepMind, 2025), and Yutori Navigator (n1) (Yutori, 2025). All OpenAI models were accessed in April and May 2026, and all leaderboard numbers were retrieved as recent as May 14, 2026.

We also compare to Set-of-Mark agents, which prompt multimodal language models using screenshots annotated with accessibility trees (Abuelsaad et al., 2024; GLM-V, 2025; He et al., 2024a; Yang et al., 2023; Zhou et al., 2023). All SoM agents share the same implementation to parse and visualize the set of marks. The agent iteratively prompts a backbone LLM to complete the task without any orchestrator. We evaluate GPT-5 (OpenAI, 2025a) and o3 (OpenAI, 2025b) as backbones.

4.1 Agentic Evaluations

4.1.1 Environment and Settings

Agentic benchmarks are evaluated against live websites whose state changes day to day, which makes reproducible comparisons difficult. In Awadallah et al. (2025), we have developed a robust evaluation protocol to help mitigate these comparison issues, which we employ in this work as well with a few updates. We refresh time sensitive tasks for each benchmark so they are current and valid. We also run our models three times and report average scores. All settings remain the same unless otherwise specified.

Browserbase. We use Browserbase to host the browser session for each task, which reduces the rate of session-level blocking on online sites.¹ To handle the small fraction of sites where Browserbase itself is unreliable, every benchmark is executed in two passes: a first pass through Browserbase, followed by a second pass that re-runs only the tasks that failed due to site-blocking without Browserbase.

Critical point handling. Fara1.5 is trained to issue `ask_user_question` at critical points (Section 2.3), but no

¹<https://www.browserbase.com/>

real user is available during automated evaluation, so to avoid wedging the trajectory in a wait-for-user state the eval harness intercepts every `ask_user_question` call from Fara1.5 and injects a fixed canned reply requesting the model to continue without hallucinating or crossing a critical point. The agent then resumes from the current browser state. The rubric judge for WebTailBench v1.5 is aware that no user simulator is wired up at eval time and consequently does not reward criteria of the form “*the agent asked the user ...*,” and the outcome judge treats properly stopping at a critical point as success rather than as a missed deferral. Baselines that have not been trained to ask the user (the SoM agents and GPT-5.4) instead have `ask_user_question` removed from their tool list altogether.

Per-benchmark judges. For each benchmark we use the official LLM judge published with that benchmark. WebVoyager uses the GPT-4o judge from [He et al. \(2024a\)](#), Online-Mind2Web uses the WebJUDGE of [Xue et al. \(2025\)](#) backed by o4-mini, and WebTailBench uses a multi-stage rubric judge from [\(Rosset et al., 2026\)](#) that combines GPT-5.2 and o4-mini, where GPT-5.2 performs the multimodal stages and o4-mini handles the action-only baseline scoring and final task verification. For WebTailBench, the rubric judge scores each trajectory against a precomputed per-task rubric with a success threshold of 0.8.

WebTailBench v1.5. The original WebTailBench ([Awadallah et al., 2025](#)) verifier penalizes agents that stop at a customer-details page rather than completing a purchase, even when completion would require the agent to fabricate PII. To avoid rewarding such fabrication, we adapt the verifier so that stopping at a critical point counts as success when the task cannot legitimately be completed without information the agent has not been given. We also refresh 270 of the original tasks to update stale dates and reduce ambiguity in the task wording. The resulting benchmark, which we call WebTailBench v1.5, is scored along two metrics: *process success*, which credits the agent for taking the correct intermediate steps up to the critical point, and *outcome success*, which additionally requires that the final task state be correct.

4.1.2 Main Results

Scores across our three browser benchmarks are reported in Table 3. We organize the table into three groups: larger and proprietary CUAs, open-weight CUAs in the same 7–9B size class as Fara1.5-9B, and our own Fara1.5 family across all three scales; the GPT-5.4-based FaraGen1.5 solver is reported separately at the bottom as an upper-bound reference.

Comparison to Fara-7B. We first compare Fara1.5-9B directly against our prior generation Fara-7B ([Awadallah et al., 2025](#)) across every benchmark we evaluate. Fara1.5-9B improves on Fara-7B on all five benchmarks, with the largest gain on Online-Mind2Web (+29.3 absolute), followed by ScreenSpot-Pro (+18.1), WebVoyager (+13.1), OSWorld-G Refined (+8.9), and WebTailBench (+8.2 on outcome success). Online-Mind2Web is the benchmark on which we close the largest absolute gap, suggesting that the synthetic-environment trajectories and the improved task proposal pipeline together address the long tail of agentic behaviors that Fara-7B struggles with.

Similarly sized agents. Compared against the open-weight CUAs in the same 7–9B parameter class, Fara1.5-9B outperforms every prior agentic SLM on both WebVoyager (86.6 vs the next best Holo2 at 80.2) and Online-Mind2Web (63.4 vs the next best GUI-Owl-1.5 at 48.6), a +14.8 absolute improvement on Online-Mind2Web over the prior state of the art at this scale.

Larger and proprietary agents. Fara1.5-27B also outperforms much larger proprietary CUAs on both benchmarks: 89.3 on WebVoyager (vs OpenAI Operator at 87.0) and 72.3 on Online-Mind2Web (vs Yutori Navigator (n1) at 64.7, OpenAI Operator at 58.3, and Google Gemini 2.5 Computer Use at 57.3). Even our 9B model is competitive with these much larger systems. As a reference, the FaraGen1.5 solver itself reaches 93.4 and 83.4 on WebVoyager and Online-Mind2Web, respectively, which provides an approximate upper bound for what the SFT-based distillation we use here can extract.

WebTailBench v1.5. On WebTailBench v1.5, Fara1.5-9B improves outcome success by +8.2 absolute over Fara-7B (24.1 to 32.3) and process success by +15.7 (48.8 to 64.5) at comparable scale, suggesting that the gains from FaraGen1.5 carry over to the long tail of web tasks under-represented in WebVoyager and Online-Mind2Web. We see similar model scaling trends on WebTailBench as we do with other benchmarks as we observed a +12.8 and +12.6 point increase in outcome and process success, respectively. Notably, Fara1.5-27B

Model	Size	WebVoyager	Online-Mind2Web	WebTailBench v1.5	
				Process	Outcome
<i>Larger and proprietary agents</i>					
o3 SoM (OpenAI, 2025b)	—	79.3	55.4	69.5	35.0
GPT-5 SoM (OpenAI, 2025a)	—	90.6	57.7	69.2	45.1
Gemini 2.5 Computer Use [†] (DeepMind, 2025)	—	—	57.3	—	—
OpenAI Operator [†] (OpenAI, 2025c)	—	87.0	58.3	—	—
Yutori Navigator (n1) [†] (Yutori, 2025)	—	—	64.7	—	—
GUI-Owl-1.5 [†] (Xu et al., 2026)	32B	82.0	—	—	—
Holo2 [†] (Company, 2025)	30B-A3B	83.0	—	—	—
<i>Similarly sized agents</i>					
Fara-7B (Awadallah et al., 2025)	7B	73.5	34.1	48.8	24.1
MolmoWeb [†] (Gupta et al., 2026)	8B	78.2	35.3	—	—
Holo2 [†] (Company, 2025)	8B	80.2	—	—	—
GUI-Owl-1.5 [†] (Xu et al., 2026)	8B	78.1	48.6	—	—
<i>Fara1.5 family (ours)</i>					
Fara1.5-4B	4B	80.8	57.3	60.3	27.4
Fara1.5-9B	9B	86.6	63.4	64.5	32.3
Fara1.5-27B	27B	89.3	72.3	72.9	40.2
FaraGen1.5 Solver (GPT-5.4)	—	93.4	83.4	79.6	57.4

Table 3: Task success rate (%) on WebVoyager, Online-Mind2Web, and WebTailBench v1.5. Higher is better. For WebTailBench v1.5 we report both *Process Success* (correct intermediate steps) and *Outcome Success* (final task state correct). **All Fara1.5 and Fara-7B numbers are averaged over three independent runs.** The GPT-5.4-based FaraGen1.5 solver is reported as an upper-bound reference for the SFT-based distillation. [†] denotes numbers sourced from the model’s official release or leaderboard rather than re-run by us. Per-benchmark 95% confidence intervals are reported in Appendix A.1.

outperforms o3 SoM and is competitive with GPT-5 SoM on WebTailBench v1.5.

Model scaling. Holding the training data and recipe fixed, we evaluate the three Fara1.5 variants on WebVoyager and Online-Mind2Web (Figure 7). Both metrics improve monotonically with parameter count: going from Fara1.5-4B to Fara1.5-27B yields +8.5 points on WebVoyager (80.8 to 89.3) and +15.0 points on Online-Mind2Web (57.3 to 72.3). Fara1.5-9B already covers a substantial fraction of the 4B-to-27B gain on both metrics, which makes it the natural choice when deployment cost is a constraint. Fara1.5-27B is the right choice when raw accuracy matters more than cost.

Number of steps versus success. We analyze if there is a relationship between length of trajectory and success on the benchmark tasks. On average, the larger and more accurate models tend to take less steps as shown in Figure 8a. However, in Figure 8b, we see that regardless of model size, there are consistent declines in success rate as trajectories get longer, which are more pronounced on the more difficult benchmarks (e.g., Online-Mind2Web or WebTailBench v1.5). These results suggest that part of what distinguishes models from each other is their *efficiency* in solving tasks, and more successful models tend to be more efficient.

4.1.3 Synthetic Environment Results

We also evaluate Fara1.5 on our synthetic environments to determine: 1) whether the tasks we generate inside a synthetic environment are actually learnable; 2) whether learning from trajectories in the synthetic environments transfers to real environments (*i.e.*, live websites). The first question is a sanity check for the validity of our synthetic environments, while the second tells us how realistic our synthetic environments are.

Held-out FaraEnvs evaluation. To answer the first question we evaluate on held-out validation tasks drawn from the six *FaraEnvs* (Section 2.1.2): Mail, Calendar, Stream, ML, Stay, and Scheduler. We compare Fara-7B, Fara1.5-9B, and the GPT-5.4-based FaraGen1.5 solver that produced the training trajectories (Table 4). Fara-7B was trained only on open-internet data and achieves an average of 18.8, confirming that generalizing from open-internet trajectories to gated domains is challenging. When trained on these environments, Fara1.5-9B reaches 71.8 average, closing most of the gap to the 79.4 of the GPT-5.4 solver, which validates that the tasks we generate are learnable within the environments and that distillation from the solver is effective.

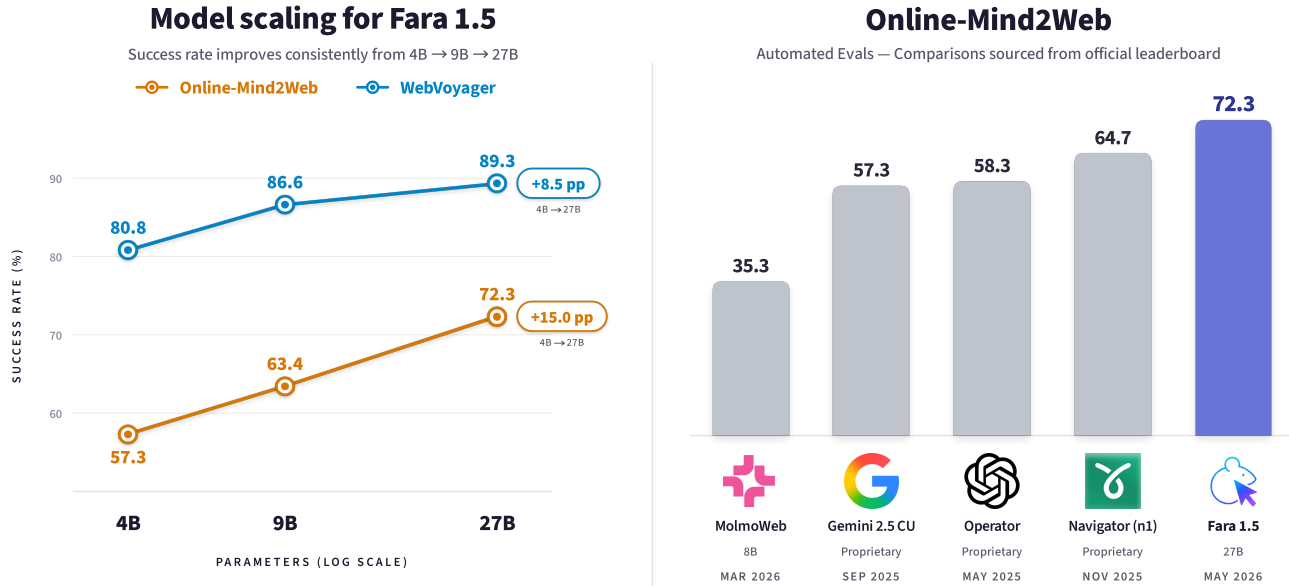


Figure 7: (Left) WebVoyager and Online-Mind2Web success rate as a function of model size for the Fara1.5 family. Both metrics improve monotonically from Fara1.5-4B to Fara1.5-27B, with +8.5 and +15.0 absolute gains respectively. (Right) Online-Mind2Web success rate of Fara1.5-27B alongside larger proprietary CUAs (sourced from the official leaderboard as of May 2026); Fara1.5-27B leads. Higher is better.

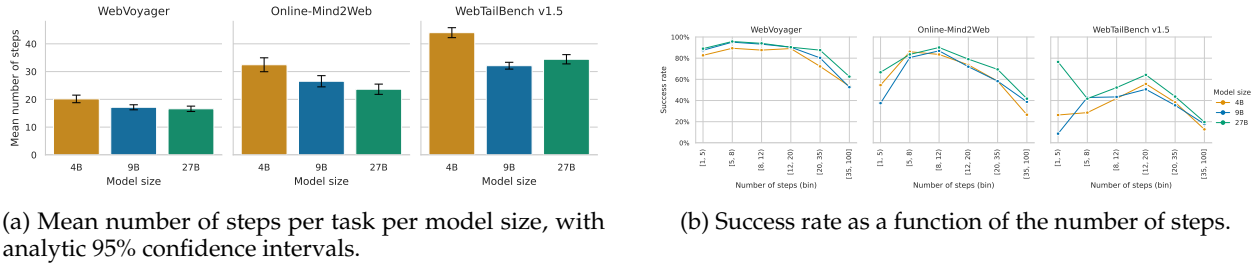


Figure 8: Plots of average number of steps per task and success rate as a function of number of steps.

Model	Mail	Calendar	Stream	ML	Stay	Scheduler	Avg.
Fara-7B	16.4	11.6	21.0	11.5	18.0	34.3	18.8
Fara1.5-9B (ours)	77.3	77.3	75.0	77.0	56.0	68.0	71.8
FaraGen1.5 solver (GPT-5.4)	81.7	81.9	76.0	86.0	75.0	76.0	79.4

Table 4: Task success rate (%) on held-out tasks from the six *FaraEnv*s synthetic environments. Fara1.5-9B outperforms Fara-7B, which has only been trained on open-internet data, and approaches the GPT-5.4-based FaraGen1.5 solver that produced the training trajectories.

Synthetic-to-real transfer. For the second question, we construct four additional synthetic environments modeled directly after four random domains covered by WebVoyager: Allrecipes, Apple, HuggingFace, and GitHub. For each one, we build a replica with our same pipeline described earlier, again prioritizing depth over breadth: rather than shallow mockups, each environment reproduces the target site’s core feature set and is populated with coherent, realistic data, so that solving a task requires engaging with the application’s actual data model and UI patterns. We then use FaraGen1.5 to generate trajectories on these replica environments (*synth-replica*), fine-tune Qwen3.5 (9B) on the resulting data, and evaluate on the corresponding *live* websites. As a control, we train an otherwise-identical baseline on a small amount of FaraGen1.5 data drawn from

Domain	Allrecipes	Apple	HuggingFace	GitHub	Combined
Baseline	87.5	68.8	59.5	75.6	73.4
+ synth-replica	92.5	81.3	73.0	85.4	83.4
Fara1.5-9B	96.7	85.4	87.4	88.6	89.8

Table 5: Synthetic-to-real transfer on four WebVoyager domains. All models use Qwen3.5 (9B) as the base. *Baseline* is fine-tuned on a small amount of data drawn from *other* FaraGen1.5 domains as a control; + *synth-replica* additionally trains on FaraGen1.5 trajectories produced inside synthetic replicas of the four target domains, and is evaluated on the corresponding live websites. Fara1.5-9B (trained on the full FaraGen1.5 mix) is shown as a reference upper bound. Higher is better.

Model	ScreenSpot-v2	ScreenSpot-Pro	OSWorld-G Refined
Fara1.5-4B	92.3	59.9 (55.3)	68.0
Fara1.5-9B	94.0	66.2 (58.0)	69.7
Fara1.5-27B	94.4	66.8 (58.6)	71.3

Table 6: Grounding accuracy (%) on ScreenSpot-v2, ScreenSpot-Pro, and OSWorld-G Refined. For ScreenSpot-Pro, we report scores without zoom in parentheses. Higher is better.

other unrelated domains simply to ensure the model can operate in our agent harness.

Table 5 shows that training on synth-replica trajectories improves the combined task success rate from 73.4 to 83.4. This improvement suggests that the synthetic environments are reasonably faithful to the real domains such that models can learn meaningful behaviors that transfer. This is despite some noticeable distribution shift between our sandboxed environments and the live website (*e.g.*, layout variations, ads, latency, dynamic content). Overall, these results imply that synthetic environments can be a useful substitute for real environments on tasks that cannot be collected on the open web.

4.2 Grounding

We evaluate on three standard grounding benchmarks: ScreenSpot-v2 (Wu et al., 2024), which covers mobile, desktop, and web; ScreenSpot-Pro (Li et al., 2025a), which focuses on high-resolution professional GUIs; and OSWorld-G Refined (Xie et al., 2025), which spans a broad set of desktop applications and also tests refusal on out-of-scope queries. Results are reported in Table 6. We find that our models are generally strong at grounding for image resolutions they have been trained on. For high-resolution images, using a two-step zoom approach shows noticeable improvements. The zoom procedure has our model first predict a coarse target region, then we crop the screenshot around this region, and finally we feed the cropped image back to the model to predict the final coordinate.

4.3 Safety

Safety for CUA models is critical as the actions of these models can have real-world consequences.

Following OpenAI (2025c) and Awadallah et al. (2025), we focus on three risk scenarios:

- **Harmful tasks:** the user requests the agent to perform a harmful task.
- **Model mistakes:** the agent inadvertently performs a harmful action while attempting a non-harmful task.
- **Harmful websites:** the agent encounters harmful content (*e.g.*, a prompt injection) while attempting a non-harmful task.

We train Fara1.5 on a mixture of public safety datasets and internally generated tasks that span the categories enumerated in Awadallah et al. (2025) (illegal activities, deceptive tasks, high-risk regulated domains, harassment and hate, irresponsible use of technology, misinformation, and sexual content), following Microsoft’s Responsible AI Policy, and we additionally train it to halt at *critical points* (Section 2.3) before taking irreversible actions.

Refusal behavior. We evaluate refusal behavior on an internal red-team suite that covers both direct harmful-task prompts and adversarial prompt-injection prompts that try to coax the agent into taking harmful actions despite a benign top-level user request. The suite is judged by a GPT-5.4-based LLM that classifies each trajectory according to whether the agent refused, attempted the harmful action, produced a harmful response, or silently answered only the benign portion of an attack. Across this suite, Fara1.5 improves over Fara-7B on direct harmful-task prompts and remains robust to adversarial injection prompts. On WebTailBench-Refusals, the 111-task safety benchmark introduced in [Awadallah et al. \(2025\)](#), the entire Fara1.5 family refuses 100% of the tasks.

Critical-point behavior. We re-run the critical-point evaluation from [Awadallah et al. \(2025\)](#) on Fara1.5 and observe no regression in the agent’s tendency to halt before irreversible actions: the stopping rate at critical points remains in line with Fara-7B across all three Fara1.5 scales. The additional capabilities added between Fara-7B and Fara1.5 have not weakened the defer-to-the-user behavior the model is trained for in Section 2.3.

While we have incorporated several safeguards, Fara1.5 is released as an experimental preview to invite hands-on exploration and feedback from the community. Improving safety and alignment of CUAs remains an active area of work for us and the broader community.

5 Related Work

Recent progress in agentic LLMs has been driven by parallel advances across several complementary dimensions. Improvements in multimodal foundation models have enabled the perceptual capabilities required for interpreting screens and graphical user interfaces (GUIs). Building on these advances, a substantial body of work has emerged on Computer Use Agents, spanning areas such as pixel-level grounding, action modeling, long-horizon planning, and reliable verification. We review related work across these directions below.

Screen understanding and Action Grounding. MLMs have improved their ability to understand screenshots and GUI elements ([Yang et al., 2025](#); [OpenAI, 2025a](#); [Anthropic, 2026](#); [DeepMind, 2026](#); [Abouelenin et al., 2025](#)). Works such as ScreenSpot ([Cheng et al., 2024](#); [Li et al., 2025b](#)), AugVis ([Xu et al., 2024](#)), OmniParser ([Lu et al., 2024](#)), GUI-Actor ([Wu et al., 2025](#)), Uground ([Gou et al., 2025](#)) and ScreenQA ([Baechler et al., 2024](#)) explore UI element grounding, question answering about screens, and general UI understanding. While these advances strengthen the perception pipeline, they do not address the multi-step control and stateful interaction required for full computer-use agents.

CUA models. Work on CUA models spans two broad paradigms, differing in choice of observation and action spaces. One class of agents use structured objects to understand the screen like DOM or accessibility tree. Environments such as WebShop ([Yao et al., 2022](#)), WebArena ([Zhou et al., 2023](#)), and VisualWebArena ([Koh et al., 2024](#)) provide agents with structured DOM trees or accessibility APIs. These abstractions simplify action selection and grounding. However, real-world websites often contain irregular DOM element markups, dynamically generated content, and visually rich layouts leading to a persistent gap between benchmark performance and real deployment ([Yutori, 2025](#)). Recent efforts circumvent this by adopting a pixel-in, action-out formulation ([Anthropic, 2024](#); [Qin et al., 2025](#); [Qwen Team, 2026](#); [Wang et al., 2025](#); [Peng et al., 2026](#)). These systems directly consume screenshots and output low-level actions such as clicks and scrolls.

Scaling Synthetic Data Generation. A recurring challenge in GUI-agent learning is the scarcity of large and diverse interaction trajectories. Existing data is often collected manually ([Deng et al., 2024](#); [Jian et al., 2026](#); [Fan et al., 2026b](#)), generated within constrained synthetic environments ([Wu et al., 2026](#); [Fan et al., 2026a](#); [Murty et al., 2025](#)), or restricted to a small set of websites ([He et al., 2024b](#)). There is also a line of work for trajectory-level synthetic data generation on real websites at scale ([Xu et al., 2025](#); [Pahuja et al., 2025](#)). In this work, we further scale synthetic trajectory generation using state-of-the-art solvers, carefully designed verifiers, and extend synthetic environments to unseen domains, including credential-gated ones which do not exist in prior synthetic-environment pipelines.

LLM as Robust Verifiers. A reliable verifier is a key prerequisite for closing the loop on computer use agents: success signals enable trajectory filtering, inference-time self-correction, and reward modeling for RL.

AgentRewardBench (Lù et al., 2025) systematically benchmarks LLM judges across five web environments. WebJudge (Xue et al., 2025) found it helpful to evaluate the agent trajectory through critical points. Rosset et al. (2026) introduce both process and outcome rewards that make the verifier more robust.

Benchmarks. Evaluating CUA models on web tasks is challenging due to the web’s non-stationarity and broader concerns such as safety and privacy. Existing benchmarks cover atomic capabilities, including screen understanding and grounding (Baechler et al., 2024; Li et al., 2025b), as well as multi-step browser interactions in environments (Yao et al., 2022; Zhou et al., 2023; Liu et al., 2024; Koh et al., 2024). Mind2Web and GAIA further move toward realistic task-driven evaluation (Deng et al., 2023; Mialon et al., 2023). However, existing benchmarks often rely on static pages, DOM-based interactions, or limited website diversity, and underrepresent multi-turn workflows, dynamic content, error recovery, and long-horizon productivity tasks. These limitations motivate WebTailBench, which evaluates CUAs on live websites under settings closer to real-world use.

6 Discussion

Safety. Computer use agents take actions with real-world consequences as they complete tasks on behalf of users. Therefore, we must ensure robust safety measures for their operations to prevent misuse, avoid unintended consequences and protect against external risks like prompt injections or online scams. Fara1.5 remains a research preview, and we continue to work on more robust mechanisms to ensure safe operation.

To mitigate misuse, we train Fara1.5 to refuse harmful tasks based on a mixture of public safety datasets and internally generated tasks abiding by Microsoft’s Responsible AI Policy. To prevent Fara1.5 from taking unintended actions, Fara1.5 has been trained to stop and ask the user at any critical points in the interaction. Critical points of the interaction occur when the task requires missing user information, or when the task itself is ambiguous, or when the task requires taking irreversible actions that were not authorized by the user.

When used with the MagenticLite interface (Microsoft Research AI Frontiers, 2026; Mozannar et al., 2025), all actions by the agent are logged and auditable allowing users to monitor task progress. The MagenticLite sandboxed browsers allow users to stop the agent at any time and provide a security boundary between the browser and the user’s machine.

For guidance on how to use our model safely, and the security considerations to be mindful of when using our model, please refer to our Model card.

Looking forward. Fara1.5 pushes the frontier of current computer use agents at their respective sizes, and we have ambitious plans to continue pushing the performance and applications of the Fara1.5 model family. We aim to expand the scope of environments that Fara1.5 can manipulate, including desktop and enterprise software. As we move to new environments, Fara1.5 will also need to perform new actions, such as interacting with the terminal and running scripts, or monitoring (Maldaner et al., 2026).

Author Contributions

Ahmed Awadallah. Senior author and overall project lead.

Sahil Gupta. Synthetic environment generation and evaluation.

Yash Lara. Program management and operations lead.

Yadong Lu. UI dragging datasets and Browserbase infrastructure.

Hussein Mozannar. Overall data generation and evaluation lead; contributed to infrastructure, training and project management.

Akshay Nambi. Synthetic environment generation and evaluation.

Zach Nussbaum. Contributed to model training, evaluation and training infrastructure and updated WebTailBench

Yash Pandya. Synthetic environment generation and evaluation.

Aravind Rajeswaran. Overall model training lead; contributed to infrastructure, data quality control, and evaluation.

Corby Rosset. Contributed to data generation including task proposal, universal verifier and updated WebTailBench.

Alexey Taymanov. Overall core infrastructure lead; contributed to evaluation and segment data generation.

Luiz do Valle. Contributed to data generation, including the user simulator and critical point verifier, and evaluation

Vibhav Vineet. Contributed to grounding evaluation and segment data generation on online environments.

Spencer Whitehead. Contributed to project direction and technical oversight, as well as infrastructure, model training, evaluation, and segment data generation.

Andrew Zhao. Contributed to grounding and QA training datasets; contributed to universal verifier, failure analysis, evaluation and WebTailBench.

All authors additionally contributed to the writing of the report.

Acknowledgments

We thank Sara Abdali, Pashmina Cameron, Adam Fourney, Ran Gal, Sarthak Harne, Michael Harrison, Rafah Hosn, Neel Joshi, Ece Kamar, John Langford, Maya Murad, Michael Sapienza, Sidhartha Sen, Pratyusha Sharma, Weili Shi, Amanda Swearngin, and Cheng Tan for their valuable help, insightful discussions, and continued support throughout this work.

We also thank members of the Microsoft Edge team (Tao Li, Jay Liu, Linjun Shou, Jingxia Xing, Javier Flores Assad, and Meghan Perez) for their close collaboration and help to improve our models.

References

- Abdelrahman Abouelenin, Atabak Ashfaq, Adam Atkinson, Hany Hassan Awadalla, Nguyen Bach, Jianmin Bao, Alon Benhaim, Martin Cai, Vishrav Chaudhary, Congcong Chen, Dongdong Chen, Dongdong Chen, Junkun Chen, Weizhu Chen, Yen-Chun Chen, Yi ling Chen, Qi Dai, Xiyang Dai, Ruchao Fan, Mei Gao, Mingcheng Gao, Amit Garg, Abhishek Goswami, Junheng Hao, Amr Hendy, Yuxuan Hu, Xin Jin, Mahmoud Khademi, Dongwoo Kim, Young Jin Kim, Gina Lee, Jinyu Li, Yunsheng Li, Chen Liang, Xihui Lin, Zeqi Lin, Meng-Jie Liu, Yang Liu, Gilsinia Lopez, Chong Luo, Piyush Madan, Vadim Mazalov, Ali Mousavi, Anh Nguyen, Jing Pan, Daniel Perez-Becker, Jacob Platin, Thomas Portet, Kai Qiu, Bo Ren, Liliang Ren, Sambuddha Roy, Ning Shang, Yelong Shen, Saksham Singhal, Subhojit Som, Xiaocheng Song, Tetyana Sych, Praneetha Vaddamanu, Shuohang Wang, Yiming Wang, Zhenghao Wang, Haibin Wu, Haoran Xu, Weijian Xu, Yifan Yang, Ziyi Yang, Donghan Yu, Ishmam Zahir, Jianwen Zhang, Li Lina Zhang, Yunan Zhang, and Xiren Zhou. 2025. Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs. *ArXiv* abs/2503.01743 (2025). <https://api.semanticscholar.org/CorpusID:276747153>
- Tamer Abuelsaad, Deepak Akkil, Prasenjit Dey, Ashish Jagmohan, Aditya Vempaty, and Ravi Kokku. 2024. Agent-E: From Autonomous Web Navigation to Foundational Design Principles in Agentic Systems. *arXiv preprint arXiv:2407.13032* (2024).
- Pranjal Aggarwal, Graham Neubig, and Sean Welleck. 2026. Gym-Anything: Turn any Software into an Agent Environment. *arXiv:2604.06126 [cs.LG]* <https://arxiv.org/abs/2604.06126>
- Anthropic. 2024. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>.
- Anthropic. 2026. Claude Opus 4.7 System Card. <https://www.anthropic.com/news/claude-opus-4-7>. System card linked from the announcement page.
- Anthropic. 2026. Claude Opus 4.8. <https://www.anthropic.com> Large Language Model.
- Anas Awadalla, Dhruva Ghosh, Aylin Akkus, Yuhui Zhang, Marianna Nezhurina, Jenia Jitsev, Yejin Choi, and Ludwig Schmidt. 2025. Gelato: From Data Curation to Reinforcement Learning: Building a Strong Grounding Model for Computer-Use Agents. <https://github.com/mlfoundations/gelato>.
- Ahmed Awadallah, Yash Lara, Raghav Magazine, Hussein Mozannar, Akshay Nambi, Yash Pandya, Aravind Rajeswaran, Corby Rosset, Alexey Taymanov, Vibhav Vineet, Spencer Whitehead, and Andrew Zhao. 2025. Fara-7B: An Efficient Agentic Model for Computer Use. *arXiv:2511.19663* (2025).
- Gilles Baechler, Srinivas Sunkara, Maria Wang, Fedir Zubach, Hassan Mansoor, Vincent Etter, Victor Carbune, Jason Lin, Jindong Chen, and Abhanshu Sharma. 2024. ScreenAI: A Vision-Language Model for UI and Infographics Understanding. *ArXiv* abs/2402.04615 (2024). <https://api.semanticscholar.org/CorpusID:267523393>
- Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Annual Meeting of the Association for Computational Linguistics*.
- H Company. 2025. Holo2 - Open Foundation Models for Navigation and Computer Use Agents.
- Google DeepMind. 2025. <https://blog.google/technology/google-deepmind/gemini-computer-use-model/>
- Google DeepMind. 2026. Gemma 4: Byte for Byte, the Most Capable Open Models. <https://deepmind.google/models/gemma/gemma-4/>.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. *arXiv:2306.06070 [cs.CL]* <https://arxiv.org/abs/2306.06070>

- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2024. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems* 36 (2024).
- Sicheng Fan, Qingyun Shi, Shengze Xu, Shengbo Cai, Tieyong Zeng, Li Ling, Yanyi Shang, and Dehan Kong. 2026a. WebFactory: Automated Compression of Foundational Language Intelligence into Grounded Web Agents. *arXiv preprint arXiv:2603.05044* (2026).
- Sicheng Fan, Rui Wan, Yifei Leng, Gaoning Liang, Li Ling, Yanyi Shang, and Dehan Kong. 2026b. WebChain: A Large-Scale Human-Annotated Dataset of Real-World Web Interaction Traces. *arXiv:2603.05295* <https://arxiv.org/abs/2603.05295>
- Aarash Feizi, Shravan Nayak, Xiangru Jian, Kevin Qinghong Lin, Kaixin Li, Rabiul Awal, Xing Han L   , Johan Obando-Ceron, Juan A. Rodriguez, Nicolas Chapados, David Vazquez, Adriana Romero-Soriano, Reihaneh Rabbany, Perouz Taslakian, Christopher Pal, Spandana Gella, and Sai Rajeswar. 2025. Grounding Computer Use Agents on Human Demonstrations. *arXiv:2511.07332* [cs.LG] <https://arxiv.org/abs/2511.07332>
- Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang, Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. 2024. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. *arXiv:2411.04468* [cs.AI] <https://arxiv.org/abs/2411.04468>
- GitHub. 2026. GitHub Copilot Documentation. <https://docs.github.com/en/copilot>. Accessed: 2026-06-16.
- GLM-V. 2025. GLM-4.5V and GLM-4.1V-Thinking: Towards Versatile Multimodal Reasoning with Scalable Reinforcement Learning. *arXiv:2507.01006* [cs.CV] <https://arxiv.org/abs/2507.01006>
- Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. 2025. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. *arXiv:2410.05243* [cs.AI] <https://arxiv.org/abs/2410.05243>
- Tanmay Gupta, Piper Wolters, Zixian Ma, Peter Sushko, Rock Yuren Pang, Diego Llanes, Yue Yang, Taira Anderson, Boyuan Zheng, Zhongzheng Ren, Harsh Trivedi, Taylor Blanton, Caleb Ouellette, Winson Han, Ali Farhadi, and Ranjay Krishna. 2026. MolmoWeb: Open Visual Web Agent and Open Data for the Open Web. *arXiv:2604.08516* [cs.CV] <https://arxiv.org/abs/2604.08516>
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024a. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. *arXiv:2401.13919* [cs.CL] <https://arxiv.org/abs/2401.13919>
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Hongming Zhang, Tianqing Fang, Zhenzhong Lan, and Dong Yu. 2024b. OpenWebVoyager: Building Multimodal Web Agents via Iterative Real-World Exploration, Feedback and Optimization. *arXiv:2410.19609* [cs.CL] <https://arxiv.org/abs/2410.19609>
- Xiangru Jian, Shravan Nayak, Kevin Qinghong Lin, Aarash Feizi, Kaixin Li, Patrice Bechard, Spandana Gella, and Sai Rajeswar. 2026. CUA-Suite: Massive Human-annotated Video Demonstrations for Computer-Use Agents. *arXiv:2603.24440* <https://arxiv.org/abs/2603.24440>
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. 2024. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. *arXiv preprint arXiv:2401.13649* (2024).
- Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. 2025a. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. *arXiv:2504.07981* [cs.CV] <https://arxiv.org/abs/2504.07981>

- Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. 2025b. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. *Proceedings of the 33rd ACM International Conference on Multimedia* (2025). <https://api.semanticscholar.org/CorpusID:277740982>
- Yang Li, Jiacong He, Xin Zhou, Yuan Zhang, and Jason Baldridge. 2020. Mapping Natural Language Instructions to Mobile UI Action Sequences. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 8198–8210.
- Junpeng Liu, Yifan Song, Bill Yuchen Lin, Wai Lam, Graham Neubig, Yuanzhi Li, and Xiang Yue. 2024. VisualWebBench: How Far Have Multimodal LLMs Evolved in Web Page Understanding and Grounding? *arXiv preprint arXiv:2404.05955* (2024).
- Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J Pal, and Siva Reddy. 2025. Agentrewardbench: Evaluating automatic evaluations of web agent trajectories. *arXiv preprint arXiv:2504.08942* (2025).
- Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. 2024. OmniParser for Pure Vision Based GUI Agent. *ArXiv abs/2408.00203* (2024). <https://api.semanticscholar.org/CorpusID:271601072>
- Matheus Kunzler Maldaner, Adam Fourney, Amanda Swearngin, Hussein Mozannar, Gagan Bansal, Maya Murad, Rafah Hosn, and Saleema Amershi. 2026. SentinelBench: A Benchmark for Long-Running Monitoring Agents. *arXiv:2606.05342 [cs.AI]* <https://arxiv.org/abs/2606.05342>
- Minesh Mathew, Viraj Bagal, Rubèn Pérez Tito, Dimosthenis Karatzas, Ernest Valveny, and C. V. Jawahar. 2021. InfographicVQA. *CoRR abs/2104.12756* (2021). *arXiv:2104.12756* <https://arxiv.org/abs/2104.12756>
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: benchmark for general ai assistants. *arXiv preprint arXiv:2311.12983* (2023).
- Microsoft Research AI Frontiers. 2026. MagenticLite, MagenticBrain, Fara1.5: An agentic experience optimized for small models. <https://www.microsoft.com/en-us/research/blog/magenticlite-magenticbrain-fara1-5-an-agentic-experience-optimized-for-small-models/>. Microsoft Research blog post.
- Hussein Mozannar, Gagan Bansal, Cheng Tan, Adam Fourney, Victor Dibia, Jingya Chen, Jack Gerrits, Tyler Payne, Matheus Kunzler Maldaner, Madeleine Grunde-McLaughlin, Eric Zhu, Griffin Bassman, Jacob Alber, Peter Chang, Ricky Loynd, Friederike Niedtner, Ece Kamar, Maya Murad, Rafah Hosn, and Saleema Amershi. 2025. Magentic-UI: Towards Human-in-the-loop Agentic Systems. *arXiv:2507.22358 [cs.AI]* <https://arxiv.org/abs/2507.22358>
- Shikhar Murty, Hao Zhu, Dzmitry Bahdanau, and Christopher D. Manning. 2025. NNetNav: Unsupervised Learning of Browser Agents Through Environment Interaction in the Wild. *arXiv:2410.02907 [cs.CL]* <https://arxiv.org/abs/2410.02907>
- OpenAI. 2025a. GPT-5 System Card. <https://cdn.openai.com/gpt-5-system-card.pdf>.
- OpenAI. 2025b. OpenAI o3 and o4-mini System Card. <https://cdn.openai.com/pdf/2221c875-02dc-4789-800b-e7758f3722c1/o3-and-o4-mini-system-card.pdf>.
- OpenAI. 2025c. Operator System Card. https://cdn.openai.com/operator_system_card.pdf.
- OpenAI. 2026. GPT-5.4 Thinking System Card. <https://deploymentsafety.openai.com/gpt-5-4-thinking/gpt-5-4-thinking.pdf>.
- OpenAI. 2026. Introducing GPT-5.4. <https://openai.com/index/introducing-gpt-5-4/>. Accessed: 2026-06-11.

- Arnold Overwijk, Chenyan Xiong, Xiao Liu, Cameron VandenBerg, and Jamie Callan. 2022. ClueWeb22: 10 Billion Web Documents with Visual and Semantic Information. *arXiv:2211.15848 [cs.IR]* <https://arxiv.org/abs/2211.15848>
- Vardaan Pahuja, Yadong Lu, Corby Rosset, Boyu Gou, Arindam Mitra, Spencer Whitehead, Yu Su, and Ahmed Hassan Awadallah. 2025. Explorer: Scaling Exploration-driven Web Trajectory Synthesis for Multimodal Web Agents. In *Findings of the Association for Computational Linguistics: ACL 2025*. 6300–6323.
- Baolin Peng, Wenlin Yao, Qianhui Wu, Hao Cheng, Xiao Yu, Rui Yang, Tao Ge, Alessandro Sordoni, Xingdi Yuan, Yelong Shen, et al. 2026. Orchard: An Open-Source Agentic Modeling Framework. *arXiv preprint arXiv:2605.15040* (2026).
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *arXiv preprint arXiv:2501.12326* (2025).
- Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. <https://qwen.ai/blog?id=qwen3.5>
- Corby Rosset, Pratyusha Sharma, Andrew Zhao, Miguel Gonzalez-Fernandez, and Ahmed Awadallah. 2026. The Art of Building Verifiers for Computer Use Agents. *arXiv preprint arXiv:2604.06240* (2026).
- The Yutori Team. 2025. Introducing Navigator. <https://yutori.com/blog/introducing-navigator>. Yutori Blog.
- Bowen Wang, Dunjie Lu, Junli Wang, Tianyi Bai, Shixuan Liu, Zhipeng Zhang, Haiquan Wang, Hao Hu, Tianbao Xie, Shuai Bai, Dayiheng Liu, Que Shen, Junyang Lin, and Tao Yu. 2026. CUA-Gym: Scaling Verifiable Training Environments and Tasks for Computer-Use Agents. *arXiv:2605.25624 [cs.AI]* <https://arxiv.org/abs/2605.25624>
- Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Juntong Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. 2025. UI-TARS-2 Technical Report: Advancing GUI Agent with Multi-Turn Reinforcement Learning. *arXiv preprint arXiv:2509.02544* (2025).
- Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, Si Qin, Lars Lidén, Qingwei Lin, Huan Zhang, Tongxing Zhang, Jianbing Zhang, Dongmei Zhang, and Jianfeng Gao. 2025. GUI-Actor: Coordinate-Free Visual Grounding for GUI Agents. *ArXiv abs/2506.03143* (2025). <https://api.semanticscholar.org/CorpusID:279118510>
- Yifan Wu, Yiran Peng, Yiyu Chen, Jianhao Ruan, Zijie Zhuang, Cheng Yang, Jiayi Zhang, Man Chen, Yenchu Tseng, Zhaoyang Yu, Liang Chen, Yuyao Zhai, Bang Liu, Chenglin Wu, and Yuyu Luo. 2026. AutoWebWorld: Synthesizing Infinite Verifiable Web Environments via Finite State Machines. *arXiv preprint arXiv:2602.14296* (2026).
- Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. 2024. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. *arXiv:2410.23218 [cs.CL]* <https://arxiv.org/abs/2410.23218>
- Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. 2025. Scaling Computer-Use Grounding via User Interface Decomposition and Synthesis. *arXiv:2505.13227 [cs.AI]* <https://arxiv.org/abs/2505.13227>
- Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, Zhiyuan Chen, Jitong Liao, Qi Zheng, Jiahui Zeng, Ze Xu, Shuai Bai, Junyang Lin, Jingren Zhou, and Ming Yan. 2026. Mobile-Agent-v3.5: Multi-platform Fundamental GUI Agents. *arXiv:2602.16855 [cs.AI]* <https://arxiv.org/abs/2602.16855>
- Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. 2025. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=EEgYUccwsV>

- Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. 2024. Aguviz: Unified Pure Vision Agents for Autonomous GUI Interaction. *arXiv preprint arXiv:2412.04454* (2024).
- Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An Illusion of Progress? Assessing the Current State of Web Agents. *arXiv:2504.01382 [cs.AI]* <https://arxiv.org/abs/2504.01382>
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *arXiv:2505.09388 [cs.CL]* <https://arxiv.org/abs/2505.09388>
- Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. 2023. Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v. *arXiv preprint arXiv:2310.11441* (2023).
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems* 35 (2022), 20744–20757.
- Yutori. 2025. The bitter lesson for web agents. <https://yutori.com/blog/the-bitter-lesson-for-web-agents>.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).

A Appendix

A.1 Agentic Evaluation Confidence Intervals

For each Fara1.5 model, all agentic results in this report are averaged over three independent evaluation runs. Table 7 reports the success rates with 95% confidence intervals. The intervals are small relative to the gaps between model sizes, confirming that the scaling trend in Table 3 is not an artifact of run-to-run noise.

Model	WebVoyager	Online-Mind2Web	WebTailBench v1.5
Fara1.5-4B	80.8 ± 2.3	57.3 ± 4.3	27.4 ± 2.8
Fara1.5-9B	86.6 ± 2.0	63.4 ± 4.3	32.3 ± 2.8
Fara1.5-27B	89.3 ± 1.8	72.3 ± 3.8	40.2 ± 3.0

Table 7: Success rate (%) of the Fara1.5 family with 95% confidence intervals, averaged over the three independent runs. For WebTailBench v1.5 we report Outcome Success.

A.2 Additional Analysis

This section collects supporting analyses of agent behavior on the three agentic benchmarks (WebVoyager, Online-Mind2Web, and WebTailBench v1.5). Throughout, the number of steps is the count of agent actions in a trajectory, and trajectories are capped at 100 steps.

Number of steps versus success. Figure 9 relates trajectory length to success. Very short trajectories tend to fail (the agent answers prematurely), success peaks at a small number of steps, and the success rate then declines steadily as trajectories grow longer. Equivalently, successful trajectories are markedly shorter than failed ones, which often run until the step budget is exhausted. This holds across model sizes as well.

Efficiency conditioned on success. The larger mean step counts of smaller models above are largely a consequence of their higher failure rate, not lower efficiency. Figure 10 shows the distribution of number of steps for *successful* trajectories on examples solved by all three sizes: conditioned on success, the trajectory-length distributions are very similar across sizes (WebTailBench v1.5 shows only a small residual gap for the 4B model).

Per-example difficulty. Figure 12 sorts examples from easiest to hardest by their estimated pass@k and shows that larger models solve more of the hard tail. The benchmarks also differ markedly in difficulty (WebVoyager easiest, WebTailBench v1.5 hardest, where even the 27B model leaves a substantial fraction of examples unsolved in three attempts). Figure 13 plots the median number of steps along the same difficulty ordering: harder examples consistently require more steps.



Figure 9: Distribution of the number of steps split by outcome, resolved by model size (split violins: left/green = correct, right/red = incorrect). The same pattern holds at every size—successful trajectories are consistently shorter while failures concentrate at longer lengths and at the 100-step cap—and the failure distributions shift toward the cap most for the smaller models.

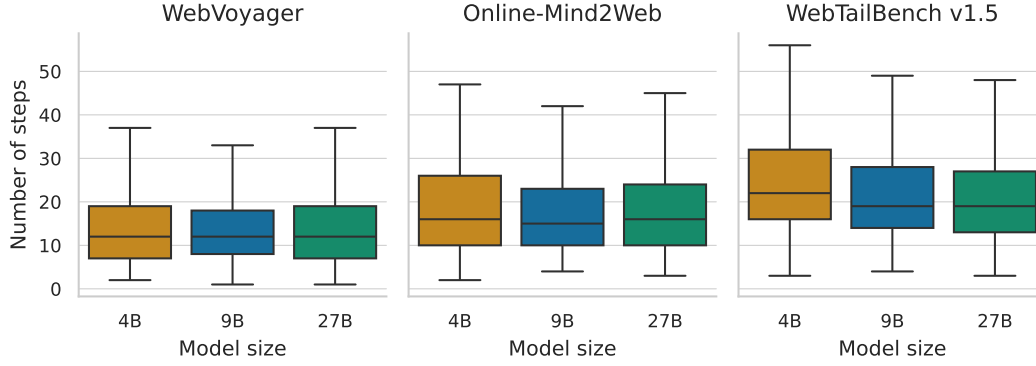


Figure 10: Number of steps for *successful* trajectories on the examples solved by all three model sizes. Conditioned on success, the distributions are nearly identical across sizes, indicating smaller models are not inherently less step-efficient.

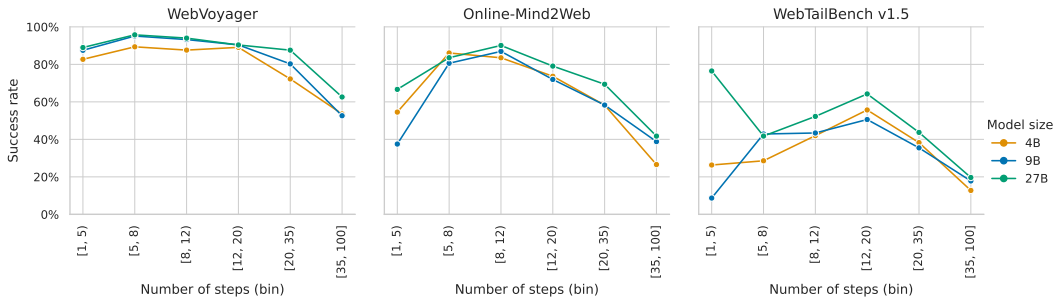


Figure 11: Success rate as a function of trajectory length, broken out by model size. The mapping is largely independent of size in the bulk of the step range; larger models retain a higher success rate only in the long tail.

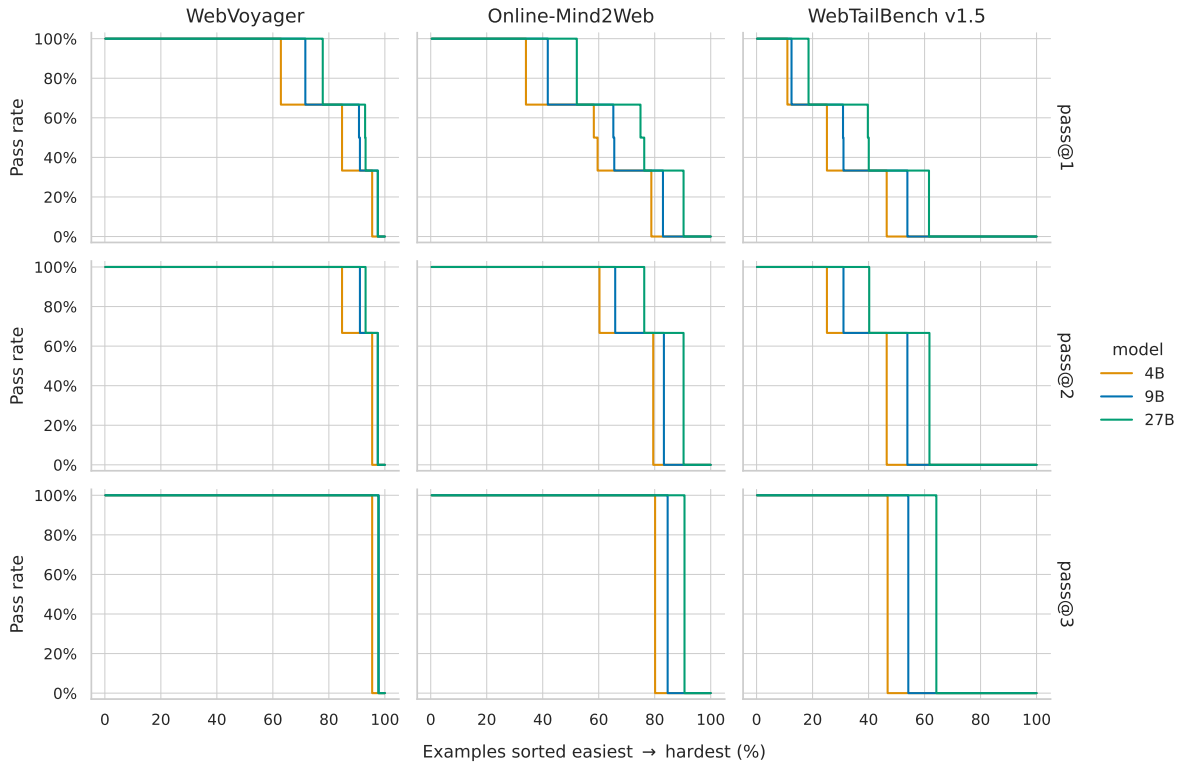


Figure 12: Per-example pass@ k ($k = 1, 2, 3$) with examples sorted from easiest to hardest. The right tail (pass rate $\rightarrow 0$) is the set of hardest examples; larger models solve more of it, and additional attempts (pass@2, pass@3) recover more examples.

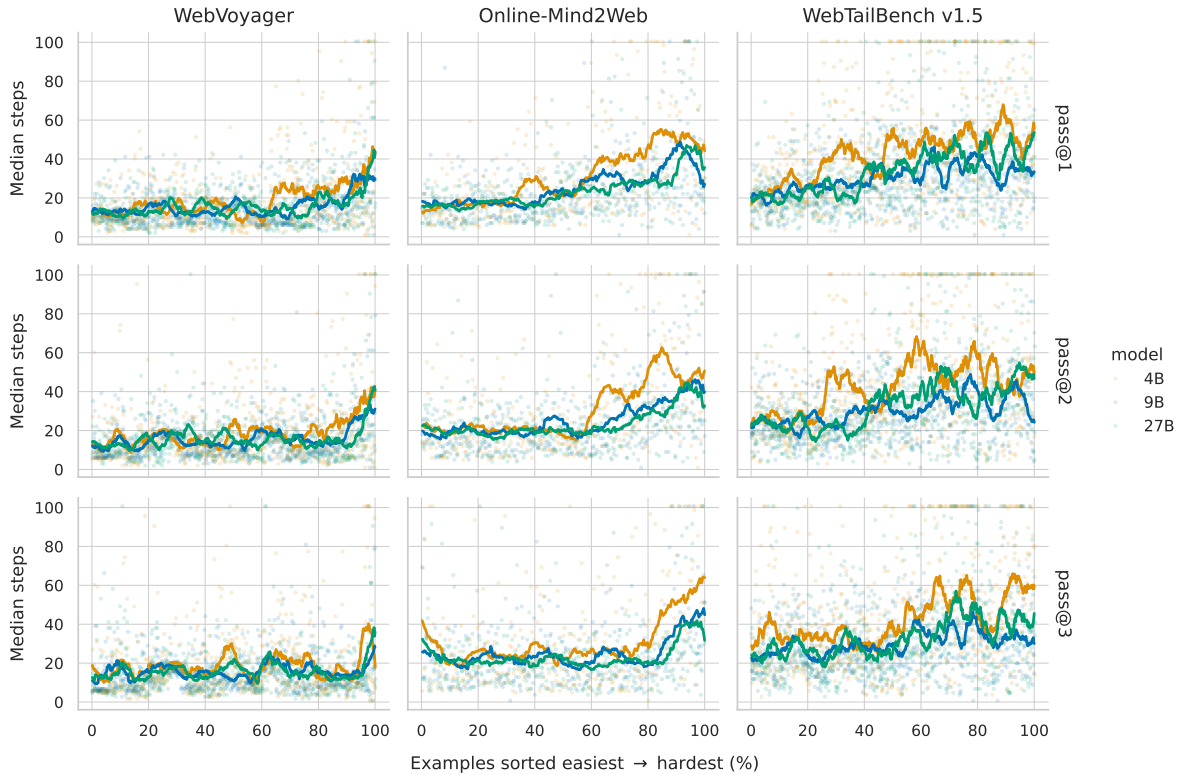


Figure 13: Median number of steps per example (faint points) and a rolling average (solid lines) along the same easiest-to-hardest ordering as Figure 12. Harder examples take more steps.

A.3 Task Proposal on Live Websites

A.4 User Simulator Prompts

The user simulator is invoked from two callsites during data generation (Section 2.2). When the solver issues an `ask_user` tool call at a critical point, we use the critical-point reply prompt below to elicit a plausible user response (personal information, preferences, or approval). Once the solver believes a task is complete, we use the follow-up generation prompt to either accept the result or extend the trajectory with a contextually grounded continuation, which is the principal source of multi-turn training data for Fara1.5.

Critical-point reply prompt (response to `ask_user`)

You are simulating a user who is interacting with a web browsing agent. The agent has reached a critical point and is requesting information or approval from you to continue. Based on the conversation history and the current screenshot, respond to the agent's request in a way to respond to the request.

Guidelines for providing information:

1. Feel free to create mock user information and make assumptions freely
2. Make sure to answer all questions and provide all necessary details
3. Make sure any assumptions you make are reasonable and consistent with the conversation history
4. If the agent asks for preferences, make a clear choice
5. If the agent needs personal details (name, email, address, etc.), provide realistic fake information

Guidelines for approval requests:

1. If the agent is requesting approval to proceed with an action (e.g., making a purchase, submitting a form), make sure it aligns with the original task
2. If the action seems reasonable and safe, approve it
3. If the action seems risky, unnecessary, or misaligned with the original task, deny it

Dimension	# values	Who decides	Values
Site	100s	sampled	popularity-weighted across 18 domain categories (e-commerce, travel, search-info, forms, food-delivery, social-media, government, finance, healthcare, news, education, real-estate, automotive, dev-tools, media-entertainment, reference-knowledge, productivity, classifieds)
Complexity	6	sampled	L1 quick action (1–5 steps) through L6 (50–100+ steps)
Phrasing	6	sampled	well-specified, formal, casual, mobile-fragment, augmented-context, ambiguous
Interaction type	7	sampled	autonomous, ambiguous, confirmation, preference, credential, correction, multi-step
Solvability	2	sampled	solvable, impossible
Intent	15	LLM chooses	12 single-site (information-extraction, site-navigation, form-completion, shopping-purchase, comparison-research, booking-reservation, ...) plus compositional (travel-planning, multi-site-procurement, research-then-act)
Persona	8	LLM chooses	busy-professional, student-budget, researcher-analyst, tourist-traveler, ...

Table 8: Task-proposal dimensions for online environments. Sampled dimensions control coverage; the proposal LLM chooses the remaining two (intent, persona) from a structured menu conditioned on the sampled site.

Site	Sampled tags	Instruction
OpenStax	education, L2, casual, correction	i need a free biology textbook pdf on OpenStax
GCP Docs	dev-tools, L1, casual, autonomous	just need the GCP Docs page for Cloud Billing
Simply Recipes	food-delivery, L2, casual, autonomous	find the Simply Recipes sweepstakes page if they have one
Movoto	real-estate, L2, mobile-fragment, preference	new listings near miami beach with pool maybe condo or house
NYTimes	news, L4, formal, confirmation	Find 3 recent NYT Wirecutter carry-on suitcase picks and compare price, weight, expandable option, and warranty. Put the best value in the cart but stop before checkout for my approval.
Hostelworld	travel, L5, casual, autonomous	find me the best 4 nights in Lisbon next month on Hostelworld for 1 person, mix of dorm and private options, then compare the top 5 by total price, rating, location, and free cancellation
Staples	e-commerce, L5, ambiguous, autonomous	need shipping and shredding info from Staples Business Center, what services are there and any obvious limits or notes
Carfax	automotive, L6, mobile-fragment, credential	carfax sell my car start cash offer for my vin ask me details when needed
UpToDate	healthcare, L6, augmented-context, multi-step	From scratch, use UpToDate to build a complete public-facing feature matrix: specialties, calculators, patient ed, drug tools, CME, apps, EHR, support. We'll refine.

Table 9: A few example tasks chosen to span all seven dimensions of variation (Table 8). Sampled tags list (domain, complexity, phrasing, interaction type) for each instruction; the proposal LLM additionally picked an intent and persona conditioned on the site.

Follow-up task generation prompt (multi-turn extension)

You are simulating a user who is interacting with a web browsing agent. Based on the conversation history and the current screenshot, generate a natural, contextually relevant follow-up task.

Generate a realistic follow-up task that:

1. Is naturally related to the conversation history and previous tasks/outputs
2. Could reasonably be asked by a user in this context
3. Leverages the current page state or information visible in the screenshot, but requires navigation beyond the current page
4. Is specific and actionable (not vague like "tell me more")
5. Requires actions (clicking, navigating to new page), beyond just reading the page
6. The follow-up task should be slightly simpler than the original task but within similar complexity
7. Do not ever refer to `https://` URLs directly in your feedback to the agent. Instead, refer to the general domain name (i.e., Google Flights) instead of an `https` URL.

Examples of good follow-up tasks:

- If the previous task was "What is the best pair of noise canceling headphones", a good follow-up: "can you see if this headphone is available on Amazon", a second follow up could be "purchase it"
- If the previous task was "Search arXiv for the latest papers on computer use agents", a good follow-up "search arxiv for latest papers on multimodal LLMs" and a second follow-up could be "find more papers on arxiv from the first author of the first paper"
- If the previous task was "find the next liverpool and boston celtics game times", a good follow-up would be "find the ticket prices for the celtics game" or "tell me what was the previous score of the last liverpool and celtics game"

Criteria:

- Tasks should work without being logged in to any accounts
- Tasks should not require irreversible actions like making a purchase (adding to cart is okay) or creating an account
- No tasks involving downloading or viewing PDFs
- Tasks should be very well specified so that by just reading the task alone one could solve it successfully
- The answer to the task should be at most a few sentences long
- Provide fake user information if needed for a follow up
- Do not hand the agent an exact deep URL. Use the site/brand name, the item or recipe name, or broad search terms instead.

A.5 Model Training Details

Table 10 summarizes the hyperparameters used to train the released Fara1.5 family. All three sizes are trained from the corresponding Qwen3.5 backbone with supervised fine-tuning under the same data mix and the same optimization schedule, so that downstream comparisons isolate the effect of model size.

Setting	Value
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-7}$)
Peak learning rate	5×10^{-6}
LR schedule	Cosine decay with warmup ratio 0.10
Epochs	2
Precision	bfloat16
Batch size	128
Training compute	32/64 $\times$ NVIDIA B200 (4/8 nodes $\times$ 8 GPUs), $\sim$ 4–5 days per run

Table 10: Training settings for the released Fara1.5 family. The 4B, 9B, and 27B variants share the optimization schedule and data mix.

A.6 Example Trajectories

We illustrate two trajectories produced by Fara1.5-9B on the WebTailBench v1.5 held-out set, both rated by the FaraBench v1.5 ensemble judge (GPT-5.2 + o4-mini). The first trajectory succeeds on both the process rubric and the outcome judgement. The second satisfies the process rubric (≥ 0.8) but is marked an outcome failure by the judge which motivates reporting both metrics in Table 3.

A.6.1 Process and outcome success: hotels_head_orbitz_8

Task. *I'd like to reserve a room at Legoland Hotel in Carlsbad, California using orbitz.com checking in October 30 - November 2. If the hotel doesn't take reservations for that date or there are no available rooms for that time, please*

indicate that in your answer.

Scores. Process rubric = 1.00; outcome judgement = **success**.

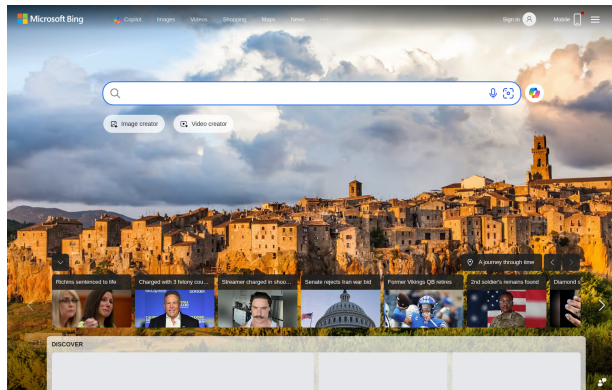
Step 1 reasoning (verbatim)

I'm starting from the Bing homepage and the task specifically names orbitz.com as the site to use. The most direct and reliable next step is to navigate straight to Orbitz rather than using Bing results, because the instructions say to complete the task within the named website's UI and to use direct navigation when the base URL is known. After loading Orbitz, I'll inspect the homepage for the hotel search form and date controls.

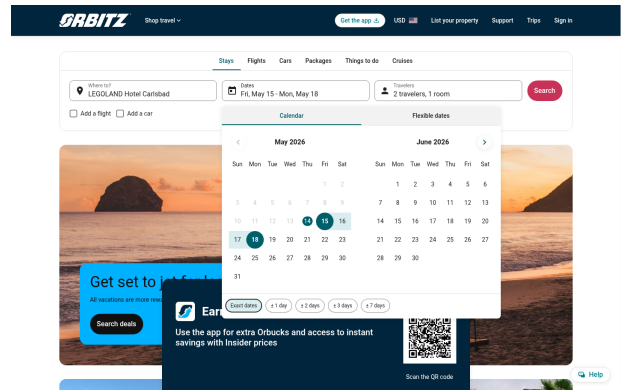
action: visit_url(url="https://www.orbitz.com/")

Full action sequence (16 steps).

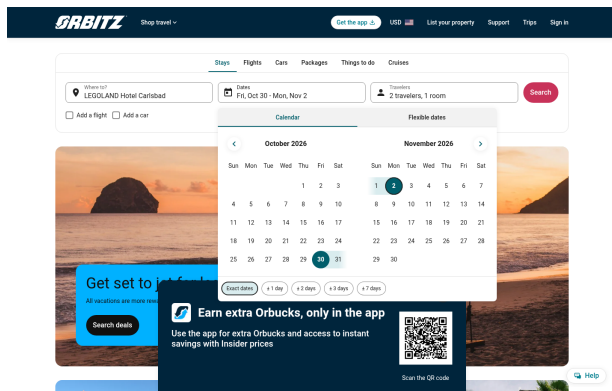
1. visit_url(<https://www.orbitz.com/>)
2. left_click(206, 221) — focus hotel-search input
3. type("LEGOLAND Hotel Carlsbad")
4. left_click(469, 222) — pick suggestion
5. left_click(769, 356) — open date picker
6. left_click(767, 357) — advance calendar
7. left_click(767, 357) — advance calendar
8. left_click(767, 357) — advance calendar
9. left_click(767, 357) — advance calendar to October
10. left_click(510, 654) — select check-in 10/30
11. left_click(628, 463) — select check-out 11/2
12. left_click(867, 222) — submit search
13. left_click(160, 567) — focus filter
14. type("LEGOLAND Hotel Carlsbad") — refine to matching property
15. left_click(185, 656) — inspect property card
16. terminate(answer="Orbitz shows 'We are sold out' for Legoland Hotel California for Oct 30–Nov 2...")



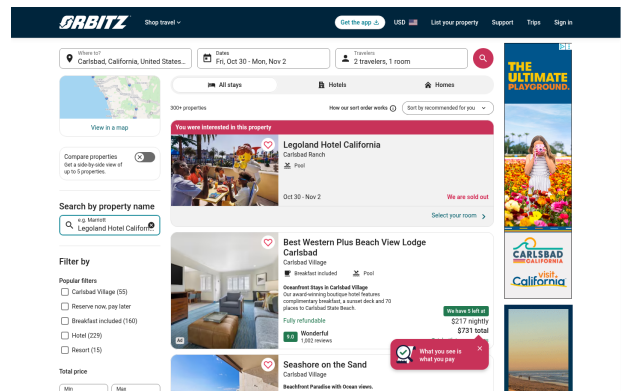
(a) Step 1: starting on Bing.



(b) Step 5: Orbitz date picker.



(c) Step 12: results refined to the property.



(d) Step 16: "We are sold out" visible on the property card.

Figure 14: Selected screenshots from `hotels_head_orbitz_8`. The agent visited the requested site, entered the hotel name and dates, and read the sold-out banner directly off the property card before terminating.

A.6.2 Process-only success: jobs_apply_apply_1219

Task. Help me apply for a chemistry research scientist position in Madison, WI found on LinkedIn and output five listings that would have me be an early application according to the listings. If there are less than five such listings, output them all.

Scores. Process rubric = 0.94 (success); outcome judgement = **failure**. The judge marks the outcome a failure because one of the five returned listings (Scientist I, Quality Control — Arrowhead Pharmaceuticals) is located in Verona, WI rather than Madison, WI, violating the explicit location constraint despite the rest of the deliverable being correct.

Step 1 reasoning (verbatim)

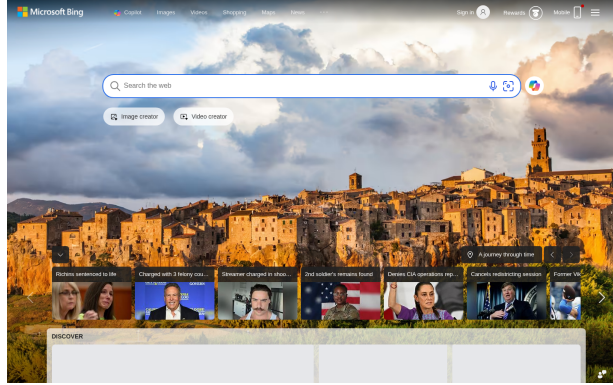
The current screen appears to be a blank browser page with no visible content loaded yet. To efficiently locate relevant LinkedIn job postings, I will navigate directly to a LinkedIn Jobs search URL that filters for chemistry research scientist roles in Madison, Wisconsin and sorts by earliest applying. This uses LinkedIn's own interface rather than an external search engine, which matches the task requirement.

```
action: visit_url(url="https://www.linkedin.com/jobs/search/?keywords=chemistry%20internship&location=United%20States&from_results_page=1")
```

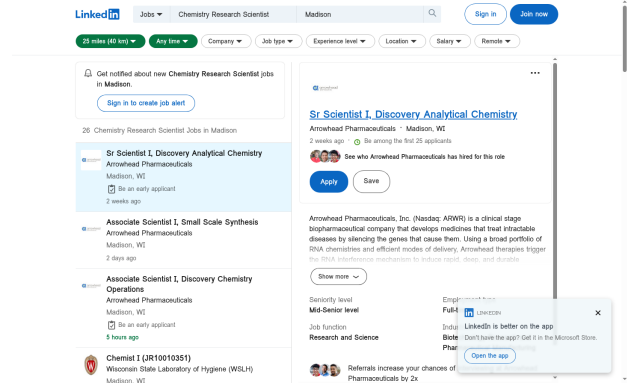
Full action sequence (8 steps).

1. `visit_url(LinkedIn Jobs search for “chemistry research scientist” in Madison, WI, sorted by date)`
2. `left_click(596, 290)` — dismiss the sign-in popup
3. `pause_and_memorize_fact(“Confirmed early-application listings so far: 1) Sr Scientist I, ...”)`
4. `scroll(-622)` — scroll down through results

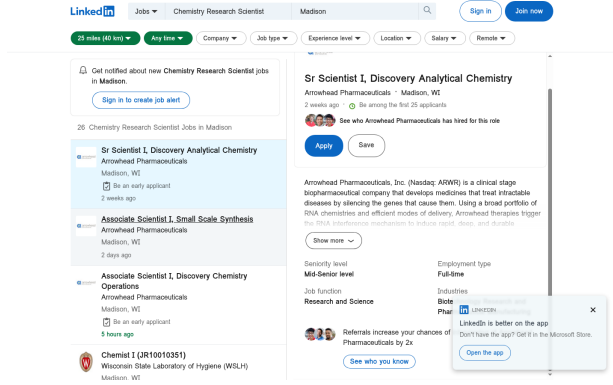
5. mouse_move(297, 607)
6. scroll(-544) — continue scrolling
7. pause_and_memorize_fact(“Additional confirmed early-application listings: 3) Chemist I (JR10010351), ...”)
8. terminate(answer=“Five postings with the ‘Be an early applicant’ label: ...”)



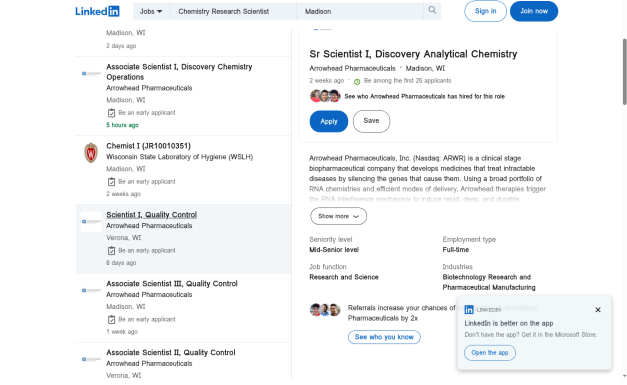
(a) Step 1: blank browser, prior to LinkedIn URL navigation.



(b) Step 4: LinkedIn results, popup dismissed, first listings visible.



(c) Step 6: mid-scroll through the results, additional early-applicant tags visible.



(d) Step 8: final state before terminate.

Figure 15: Selected screenshots from jobs_apply_apply_1219. The trajectory follows a clean process: navigate to the filtered LinkedIn search, dismiss the modal, collect candidates via pause_and_memorize_fact while scrolling. The outcome judge nevertheless marks this a failure because one returned listing is located in Verona, WI rather than the Madison, WI specified by the task.

A.7 Failure Analysis

Expanding on the failure analysis from Rosset et al. (2026), we provide an error taxonomy of common errors we observed in our trajectories in developing our data generation and evaluation pipeline in Table 11. We use this error taxonomy along with the trajectory history and rich evaluated rubric of the Universal Verifier to determine all points of failure within the trajectory. We provide a distribution of failures of WebVoyager, Online-Mind2Web, and WebTailBench in Tables 12, 13, and 14, respectively.

Error Type	Description
1. Selection	
1.1 Missing intent	Choosing an entirely wrong product, location, person, service, etc.
1.2 Unauthorized substitution	Silently swapping an unavailable item for a similar alternative without reporting
1.3 Wrong action type	Performing the wrong interaction on the correct entity
1.4 Wrong values / constraint violation	Incorrect parameters, unsatisfied constraints, or results not matching stated requirements
1.5 Other	Selection error not covered above
2. Hallucination	
2.1 Output contradiction	Evidence shows X, but agent claims not-X; includes misinterpreting page/tool content
2.2 Action contradiction	Agent claims action was performed but evidence contradicts; action was achievable
2.3 Output fabrication	Agent claims a fact with zero evidentiary basis; complete invention
2.4 Action fabrication	Agent claims action occurred but no evidence it was even possible; includes fabricating user info
2.5 Other	Hallucination error not covered above
3. Execution & Strategy	
3.1 Computational mistakes	Correct methodology but wrong answer due to miscounting, arithmetic, or misreading
3.2 Platform non-compliance	Not attempting the specified platform or silently switching sources
3.2.1 API-Sniffing	Agent navigates to a site's underlying JSON/REST API instead of the GUI URL the task implied, when the task / platform required GUI use or the bypass skips an auth or critical-point gate
3.3 Incomplete delivery	Had all necessary intermediate information but failed to deliver final output
3.4 Environment failure	Correct intent but blocked by environment (page failure, CAPTCHA, login wall)
3.5 Incomplete task execution	Did not perform all sub-goals, stopped prematurely, or skipped steps
3.6 Other	Execution error not covered above
4. Critical Point	
4.1 Premature stop	Stopped at critical point despite user explicitly granting permission
4.2 Violation	Crossed transactional boundary without permission
4.3 Other	Critical point error not covered above
5. Side-Effect	
5.1 Unsolicited	Any lasting modification, enrollment, or addition not requested
5.2 Other	Side-effect error not covered above
6. Tool Interaction	
6.1 Invalid invocation	Tool call with wrong arguments (action exists but args are incorrect)
6.2 Hallucinated action	Agent invokes a tool/action that does not exist in the action space
6.3 Intent-action mismatch	Agent's stated intent differs from actual tool call issued
6.4 Grounding error	Correct target identified but (x, y) coordinates do NOT land on the target element
6.5 Grounding intent-action mismatch	Agent's intent references an object/element that does not exist on the current screenshot
6.6 Other	Tool interaction error not covered above

Table 11: Error taxonomy for computer-use agent failures.

Code	Error type	Fara1.5-4B	Fara1.5-9B	Fara1.5-27B	GPT5.4
1	Selection Errors	27.8%	24.6%	18.3%	14.1%
1.1	Missing Intent	0.8%	0.6%	0.4%	0.1%
1.2	Unauthorized substitution	0.1%	0.1%	0.3%	0.0%
1.3	Wrong action type	1.3%	0.8%	0.5%	1.1%
1.4	Wrong values or constraint violation	26.6%	23.5%	17.7%	13.1%
1.5	Other	0.2%	0.2%	0.1%	0.3%
2	Hallucination Errors	46.8%	42.5%	37.8%	22.0%
2.1	Output contradiction	34.5%	32.1%	26.8%	11.3%
2.2	Action contradiction	1.1%	0.7%	1.0%	0.3%
2.3	Output fabrication	18.4%	17.5%	15.4%	11.5%
2.4	Action fabrication	1.2%	0.6%	1.0%	0.4%
2.5	Other	0.1%	0.1%	0.1%	0.0%
3	Execution & Strategy Errors	71.6%	69.1%	66.6%	74.6%
3.1	Computational mistakes	2.2%	1.4%	1.3%	0.3%
3.2	Platform non-compliance	2.0%	2.5%	1.4%	0.7%
3.2.1	API-Sniffing	0.7%	0.6%	0.2%	1.7%
3.3	Incomplete delivery	13.2%	10.8%	8.8%	6.4%
3.4	Environment failure	3.0%	2.7%	2.7%	2.4%
3.5	Incomplete task execution	68.1%	65.4%	62.9%	72.2%
3.6	Other	5.7%	4.0%	5.1%	2.9%
4	Critical Point Errors	0.2%	0.2%	0.1%	0.3%
4.2	Critical Point violation	0.2%	0.2%	0.1%	0.2%
4.3	Other	0.0%	0.0%	0.0%	0.1%
5	Unsolicited Side-Effect Errors	0.1%	0.1%	0.1%	0.0%
5.1	Unsolicited side effects	0.1%	0.1%	0.1%	0.0%
6	Tool Interaction Errors	25.9%	23.8%	18.5%	19.4%
6.1	Invalid invocation	0.2%	0.1%	0.0%	0.0%
6.3	Intent-action mismatch	1.3%	0.9%	1.4%	0.3%
6.4	Fine-grained grounding error	24.8%	23.2%	17.5%	17.1%
6.5	Grounding intent-action mismatch	0.1%	0.2%	0.1%	2.5%
6.6	Other	0.1%	0.2%	0.1%	0.0%
–	Total Error Segments	2970	2268	1788	1607

Table 12: WebVoyager error prevalence. A trajectory contributes to the counts if the judge determines there is an error within that trajectory. Cells are heat-shaded by prevalence. Zero-everywhere subcodes and categories are omitted.

Code	Error type	Fara1.5-4B	Fara1.5-9B	Fara1.5-27B	GPT5.4
1	Selection Errors	38.7%	35.5%	29.4%	18.1%
1.1	Missing Intent	1.4%	1.0%	0.8%	0.0%
1.2	Unauthorized substitution	0.8%	0.1%	0.0%	0.0%
1.3	Wrong action type	2.0%	1.7%	2.4%	1.2%
1.4	Wrong values or constraint violation	36.2%	33.8%	27.5%	17.0%
1.5	Other	0.3%	0.2%	0.0%	0.0%
2	Hallucination Errors	53.6%	52.2%	52.2%	24.7%
2.1	Output contradiction	41.2%	39.1%	41.2%	15.2%
2.2	Action contradiction	1.1%	0.8%	1.2%	0.3%
2.3	Output fabrication	19.6%	19.4%	16.9%	9.8%
2.4	Action fabrication	2.1%	1.3%	1.6%	0.5%
2.5	Other	0.3%	0.1%	0.0%	0.0%
3	Execution & Strategy Errors	82.7%	81.8%	79.2%	82.4%
3.1	Computational mistakes	2.1%	1.9%	0.8%	0.1%
3.2	Platform non-compliance	3.7%	3.6%	2.4%	1.7%
3.2.1	API-Sniffing	0.2%	0.9%	0.4%	0.9%
3.3	Incomplete delivery	24.7%	24.4%	20.8%	12.5%
3.4	Environment failure	4.6%	3.6%	3.5%	4.1%
3.5	Incomplete task execution	78.9%	76.3%	74.9%	80.3%
3.6	Other	8.7%	8.9%	5.9%	3.3%
4	Critical Point Errors	1.5%	1.4%	0.8%	1.4%
4.1	Premature stop (with permission)	0.1%	0.0%	0.0%	0.0%
4.2	Critical Point violation	1.3%	1.4%	0.8%	1.4%
5	Unsolicited Side-Effect Errors	0.0%	0.0%	0.0%	0.1%
5.1	Unsolicited side effects	0.0%	0.0%	0.0%	0.1%
6	Tool Interaction Errors	46.5%	47.7%	32.2%	34.8%
6.1	Invalid invocation	0.3%	0.0%	0.4%	0.0%
6.3	Intent-action mismatch	1.9%	2.6%	1.6%	0.3%
6.4	Fine-grained grounding error	45.4%	46.8%	31.0%	31.0%
6.5	Grounding intent-action mismatch	0.9%	0.1%	0.0%	4.2%
6.6	Other	0.3%	0.3%	0.4%	0.0%
–	Total Error Segments	2424	2120	1380	1261

Table 13: Online-Mind2Web error prevalence. A trajectory contributes to the counts if the judge determines there is an error within that trajectory. Cells are heat-shaded by prevalence. Zero-everywhere subcodes and categories are omitted.

Code	Error type	Fara1.5-4B	Fara1.5-9B	Fara1.5-27B	GPT5.4
1	Selection Errors	42.4%	38.3%	36.5%	26.9%
1.1	Missing Intent	1.1%	0.5%	0.5%	0.9%
1.2	Unauthorized substitution	1.9%	1.4%	1.6%	0.4%
1.3	Wrong action type	0.8%	0.5%	1.8%	0.9%
1.4	Wrong values or constraint violation	40.7%	37.0%	34.7%	25.9%
1.5	Other	0.4%	0.0%	0.2%	0.3%
2	Hallucination Errors	64.7%	64.6%	56.5%	28.6%
2.1	Output contradiction	50.8%	49.9%	43.1%	19.2%
2.2	Action contradiction	1.3%	1.1%	0.7%	0.1%
2.3	Output fabrication	21.4%	21.9%	21.1%	10.8%
2.4	Action fabrication	6.6%	3.9%	2.3%	0.2%
2.5	Other	0.0%	0.2%	0.4%	0.2%
3	Execution & Strategy Errors	82.0%	82.9%	78.0%	92.7%
3.1	Computational mistakes	1.5%	2.3%	2.3%	0.5%
3.2	Platform non-compliance	2.6%	3.7%	2.9%	2.6%
3.2.1	API-Sniffing	0.2%	0.2%	0.4%	1.0%
3.3	Incomplete delivery	9.2%	8.8%	7.8%	8.0%
3.4	Environment failure	11.8%	10.1%	7.8%	12.2%
3.5	Incomplete task execution	77.7%	77.8%	74.0%	91.5%
3.6	Other	6.2%	6.9%	5.1%	11.8%
4	Critical Point Errors	3.2%	2.8%	0.7%	0.4%
4.2	Critical Point violation	3.0%	2.8%	0.7%	0.3%
4.3	Other	0.2%	0.0%	0.0%	0.1%
5	Unsolicited Side-Effect Errors	0.8%	1.6%	1.6%	0.4%
5.1	Unsolicited side effects	0.8%	1.6%	1.6%	0.4%
6	Tool Interaction Errors	36.6%	38.6%	32.5%	37.2%
6.1	Invalid invocation	0.0%	0.2%	0.0%	0.0%
6.3	Intent-action mismatch	1.9%	1.4%	1.6%	1.3%
6.4	Fine-grained grounding error	35.6%	38.1%	31.2%	31.0%
6.5	Grounding intent-action mismatch	0.0%	0.2%	0.0%	7.1%
6.6	Other	0.0%	0.0%	0.2%	0.0%
–	Total Error Segments	3119	3238	2848	3846

Table 14: WebTailBench error prevalence. A trajectory contributes to the counts if the judge determines there is an error within that trajectory. Cells are heat-shaded by prevalence. Zero-everywhere subcodes and categories are omitted.

A.8 Grounding Examples

The quality of the open source datasets are not perfect. For instance, some segments in Jedi have false positive rates of 7/10, where a false positive is defined as the Jedi dataset says an object described in the instruction exists at a coordinate in the image, but in reality it exists elsewhere. We illustrate some of the poor examples in Figures 16 and 17.

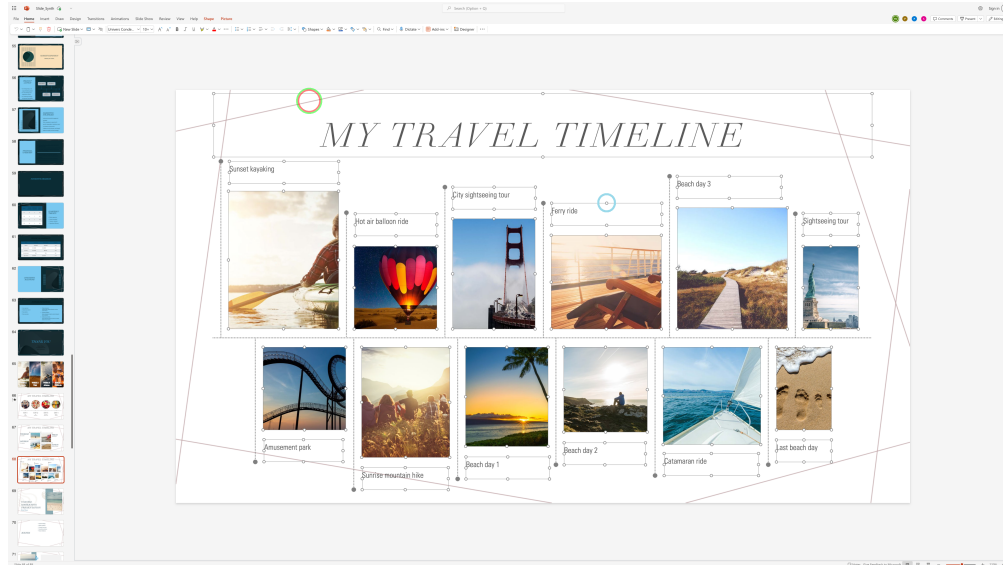


Figure 16: Task: Please generate the next move according to the UI screenshot and instruction. Instruction: Select the handle located at the top of the text box containing the text "Ferry ride." The concentric red and lime green circle denotes where the Jedi dataset **incorrectly** says the object of interest is located, the blue circle is the **correct** location of the object of interest. This is an **inaccurate** error.

Header	Name	Spend	Clicks	Metric 1	Metric 2	Metric 3	Metric 5	Metric 6	Metric 7	Estimated conversion
DT-Brand-campaign		\$1,718.56	12345	12	\$1,049.14	\$110.20	3	\$1,355.01	\$652.08	
new_offer_8735		\$1,571.63	12345	12	\$1,049.14	\$115.50	8	\$1,109.25	\$454.21	
spring_2020		\$956.54	12345	12	\$1,049.14	\$156.96	9	\$1,008.71	\$180.37	
UK_brand_DT_campaign		\$1,133.69	677	73	\$194.49	\$69.46	11	\$898.23	\$638.98	
AU-DT-offer-2394		\$1,672.93	248	15	\$190.65	\$247.49	4	\$1,580.82	\$949.96	
GR-DT-brand-12		\$704.13	2,300	12	\$104.94	\$177.04	6	\$886.22	\$68.49	
Second-offer-EU		\$1,694.09	145	42	\$116.3	\$195.97	16	\$1,653.48	\$105.08	
Follow-Far-Teams-2020		\$1,501.84	805	11	\$214.19	\$226.38	11	\$956.13	\$77.12	
Resubscription-offer-20		\$1,542.24	1,076	8	\$91.24	\$113.12	9	\$1,547.62	\$132.12	
DT-Brand-campaign-main		\$1,568.37	2,001	21	\$79.51	\$103.56	14	\$1,741.22	\$119.71	
Dave-Portnoy-campaign-20		\$1,493.87	1,706	37	\$118.74	\$75.39	8	\$1,604.28	\$102.95	
Sport-aud-second-offer		\$704.83	1,007	18	\$184.69	\$227.06	2	\$1,252.99	\$71.62	
IT-DT-brand-19-2		\$1,054.87	1,885	15	\$157.01	\$213.91	7	\$1,150.07	\$125.53	
DT-Brand-campaign-test		\$856.53	1,461	6	\$164.17	\$102.58	19	\$1,049.14	\$132.95	

Figure 17: Task: The Text field within a table cell.'s intended function: The primary function of this element is to display a specific metric value, likely representing a financial figure or performance metric. Users can view this value to assess or compare it with other metrics in the table. This is an error because the task is **not unique** for the instruction's description of the table cell.

A.9 Form-Filling Dataset

A large part of the form-filling and user-interaction slice of the training mix (Figure 4) comes from agent trajectories that fill real *Tally*² forms. Every task instantiates one of the eight critical-point types in Table 1, so the data teaches the model to complete forms while deferring to the user when information is missing, the task is underspecified, or an irreversible submission is unauthorized. Figure 18 shows the pipeline: we seed tasks from real Tally forms and critical-point prompts with synthetic user information, solve them with the GPT-5.4 solver and a GPT-5.4 user simulator that supplies missing details or permission on request, and keep only trajectories that pass the FaraGen1.5 verifiers (Section 2.3). Table 15 summarizes the result.

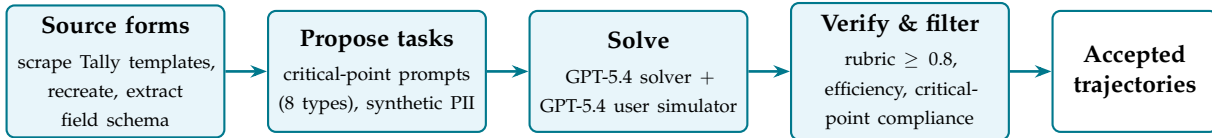


Figure 18: Form-filling data pipeline. Tasks are seeded from real Tally forms and critical-point prompts, solved by the GPT-5.4 solver paired with a GPT-5.4 user simulator that supplies missing information or permission on request, and filtered by the same verifiers as FaraGen1.5 (Section 2.3).

Property	Value
Source forms (Tally)	133
Verified trajectories	8,505
train / test	8,079 / 426
Mean steps per trajectory	27.3
Multi-turn (agent asks the user)	71%

Table 15: Fara1.5 form-filling dataset summary. “Multi-turn” is the fraction of trajectories with at least one ask_user_question.

Why we added it. Form filling exercises behaviors—entering PII and taking irreversible actions—that cannot be collected safely on the open web. On the held-out split where the user grants permission and provides all information (so the agent should simply fill and submit), Fara-7B scores 0%: it refuses nearly every form as a privacy risk *even under the new critical-point system prompt*, which it ignores. A model trained on this data reaches roughly 80% on the same split.

Worked example. Figure 19 traces a representative trajectory where the user authorizes submission but does not provide a value for one required field.

Task. Please submit [Tally form] for Olga Petrova. Make it partly anonymous (team lead can know). I’m Amanda Foster. Rate everything as Meets expectations, except Innovation as Unacceptable.

The agent enters the reviewer and reviewee, sets partial anonymity, and applies the requested ratings. It then reaches a required open-ended field, “What can Olga Petrova do better to improve?”, for which the user gave no value. Rather than inventing feedback about a real person, the agent asks:

Agent ask_user_question (verbatim)

The form has a required question: “What can Olga Petrova do better to improve?” What exact comment would you like me to enter there? If you want, you can give me a short sentence or bullet points, and I’ll submit the form after entering it.

The simulated user supplies the comment, the agent enters it verbatim, and because permission to submit was already granted it submits without asking again, reaching the confirmation page.

²<https://tally.so>

Figure 19 illustrates a form-filling trajectory across three stages:

- (a) Initial State:** The form titled "help the group achieve its goals" has four radio button options: "Exceeds expectations", "Meets expectations" (selected), "Needs improvement", and "Unacceptable". Below these is a text input field labeled "Any comments about this?". The question "What can Olga Petrova do better to improve?" is followed by another empty text input field. At the bottom, there is a section for "Any other feedback that you'd like to share?" with an optional text input and a "Submit" button.
- (b) User Input:** The user has entered text in the "Any comments about this?" field. The text reads: "Olga could improve by being more proactive in proposing new ideas and solutions, showing stronger independent problem-solving, and taking more initiative to drive improvements without waiting for direction." The "Submit" button remains visible.
- (c) Confirmation Page:** The form is completed, showing a blue checkmark icon and the message "Thanks for completing this form!". Below this, it says "Made with Tally, the simplest way to create forms for free." and provides a link to "Create your own form".

(a) The agent applies the ratings the user specified, but the required open-ended field has no provided value.

(b) Rather than fabricating it, the agent asks the user and enters the supplied comment verbatim.

(c) Since permission was already granted, the agent submits without asking again and reaches the confirmation page.

Figure 19: A form-filling trajectory where the user authorizes submission but omits a required field. The agent fills every value it was given, asks for the one missing required answer instead of fabricating it, and then submits without a redundant permission request.